

**UNIVERSITÉ
IBN TOFAIL**

Faculté des Sciences — Kénitra

SEWS MAROC

Sumitomo Electric Wiring Systems

UNIVERSITÉ IBN TOFAIL — FACULTÉ DES SCIENCES — KÉNITRA

MÉMOIRE DE FIN D'ÉTUDES

Pour l'obtention du diplôme de

MASTER SCIENCES ET TECHNIQUES

Spécialité : **Génie Informatique**

**Conception et Implémentation d'un Système Andon
Intelligent basé sur l'IIoT pour la Supervision en
Temps Réel et la Gestion du Cycle de Vie
des Pannes Industrielles**

Présenté par :

M. Aissam MALKOUT

Encadré par :

Encadrant académique : Pr. Kaoutar ALLABOUCHE

Faculté des Sciences, Université Ibn Tofail

Encadrant industriel : M. Mohamed EDDOUCHE

Responsable Maintenance, SEWS Maroc

Stage effectué à :

SEWS Maroc — Département Maintenance

Atelier Test Électrique

Kénitra, Maroc

Année Universitaire 2025–2026

*À mes chers parents,
pour leur soutien inconditionnel et leurs sacrifices,*

*À ma famille,
pour leurs encouragements constants,*

*À mes enseignants,
pour leur savoir et leur dévouement,*

*À tous ceux qui m'ont accompagné
tout au long de ce parcours académique.*

Remerciements

Ce mémoire de fin d'études, couronnement de deux années de formation intensive en Master Sciences et Techniques, n'aurait pu aboutir sans le concours et le soutien de nombreuses personnes que je tiens à remercier sincèrement.

Je tiens en premier lieu à exprimer ma profonde gratitude envers mon encadrant académique, **Pr. Kaoutar ALLABOUCHE**, pour son encadrement rigoureux, ses conseils avisés et sa disponibilité constante. Ses remarques pertinentes et son expertise scientifique ont été déterminantes pour l'orientation et la qualité de ce travail.

Mes remerciements s'adressent également à mon encadrant industriel, **M. Mohamed EDDOUCHE**, Responsable Maintenance chez SEWS Maroc, pour sa confiance, son accompagnement professionnel et le partage de son expertise technique. Je remercie également **M. Hamza BAHLOULI** pour son support technique précieux et sa patience lors des phases d'implémentation.

Je souhaite remercier l'ensemble du personnel de **SEWS Maroc**, particulièrement l'équipe du département Maintenance et de l'Atelier Test Électrique, pour leur accueil chaleureux, leur collaboration active et leur contribution à la réussite de ce projet.

Mes remerciements vont également aux membres du jury qui ont accepté d'évaluer ce travail et de l'enrichir par leurs remarques constructives.

Je remercie l'administration de la **Faculté des Sciences de l'Université Ibn Tofail** et l'ensemble du corps professoral du département Génie Informatique pour la qualité de la formation dispensée et les compétences académiques et professionnelles acquises durant ce cursus.

Enfin, je ne saurais oublier de remercier ma famille pour leur soutien indéfectible, leurs encouragements permanents et leur patience tout au long de mon parcours universitaire.

Aissam MALKOUT
Kénitra, juillet 2026

Résumé

Dans le contexte actuel de transformation digitale de l'industrie manufacturière, marqué par l'émergence de l'Industrie 4.0 et de l'Internet Industriel des Objets (IIoT), la gestion proactive et efficiente des pannes constitue un enjeu stratégique majeur pour les entreprises du secteur automobile. Le système Andon, hérité des principes du Lean Manufacturing et du Toyota Production System, permet une détection visuelle rapide des anomalies sur les lignes de production. Toutefois, ce système classique demeure limité en matière de traçabilité numérique, d'analyse temps réel et de calcul automatisé des indicateurs de performance maintenance.

Ce projet de fin d'études, réalisé au sein du département Maintenance de **SEWS Maroc** (Sumitomo Electric Wiring Systems), leader mondial du câblage automobile, vise à concevoir et développer un **système Andon intelligent basé sur l'IIoT** pour la supervision en temps réel et la gestion du cycle de vie complet des pannes industrielles dans l'Atelier Test Électrique.

La solution proposée repose sur une architecture distribuée multi-couches intégrant des capteurs connectés (**ESP32**), un protocole de communication industriel léger (**MQTT**), un backend applicatif développé en **Node.js** et déployé sur la plateforme cloud **Railway**, une base de données relationnelle **PostgreSQL**, un dashboard web temps réel exploitant les WebSockets, ainsi qu'une Progressive Web App (**PWA**) mobile destinée aux techniciens de maintenance.

Le système développé couvre l'ensemble du cycle de vie d'une panne : détection automatique via boutons Andon connectés, notification temps réel avec latence inférieure à 2 secondes, intervention tracée par identification RFID des techniciens, enregistrement structuré des observations et actions correctives, résolution automatisée et calcul instantané des indicateurs clés de performance (**MTTA** — Mean Time to Acknowledge, **MTTR** — Mean Time to Repair, disponibilité des équipements).

Les tests fonctionnels et de performance ont confirmé la fiabilité de la solution avec une latence moyenne de 520 millisecondes, une précision des calculs KPI conforme aux attentes, et une architecture scalable capable de supporter jusqu'à 100 machines et 50 utilisateurs simultanés. Ce projet démontre la faisabilité technique et la valeur ajoutée d'une approche IIoT pour moderniser les systèmes de maintenance industrielle.

Mots-clés : Système Andon, Internet Industriel des Objets (IIoT), Industrie 4.0, MQTT, Maintenance Intelligente, ESP32, Node.js, Progressive Web App, SEWS Maroc, MTTA, MTTR.

Abstract

In the current context of digital transformation within the manufacturing industry, characterized by the emergence of Industry 4.0 and the Industrial Internet of Things (IIoT), proactive and efficient breakdown management represents a critical strategic challenge for automotive sector companies. The Andon system, inherited from Lean Manufacturing principles and the Toyota Production System, enables rapid visual detection of anomalies on production lines. However, this traditional system remains limited in terms of digital traceability, real-time analysis, and automated calculation of maintenance performance indicators.

This Master's thesis project, conducted within the Maintenance Department of **SEWS Morocco** (Sumitomo Electric Wiring Systems), a global leader in automotive wiring systems, aims to design and develop an **intelligent IIoT-based Andon system** for real-time monitoring and complete lifecycle management of industrial breakdowns in the Electrical Test Workshop.

The proposed solution relies on a distributed multi-layer architecture integrating connected sensors (**ESP32**), a lightweight industrial communication protocol (**MQTT**), an application backend developed in **Node.js** and deployed on the **Railway** cloud platform, a **PostgreSQL** relational database, a real-time web dashboard leveraging WebSockets, and a mobile Progressive Web App (**PWA**) designed for maintenance technicians.

The developed system covers the entire breakdown lifecycle : automatic detection via connected Andon buttons, real-time notification with latency below 2 seconds, RFID-traced technician intervention, structured recording of observations and corrective actions, automated resolution, and instantaneous calculation of key performance indicators (**MTTA** — Mean Time to Acknowledge, **MTTR** — Mean Time to Repair, equipment availability).

Functional and performance testing confirmed the solution's reliability with an average latency of 520 milliseconds, KPI calculation accuracy meeting expectations, and a scalable architecture capable of supporting up to 100 machines and 50 simultaneous users. This project demonstrates the technical feasibility and added value of an IIoT approach to modernize industrial maintenance systems.

Keywords : Andon System, Industrial Internet of Things (IIoT), Industry 4.0, MQTT, Smart Maintenance, ESP32, Node.js, Progressive Web App, SEWS Morocco, MTTA, MTTR.

Table des matières

Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Abréviations	viii
Introduction Générale	1
1 Contexte Général et Cadre du Projet	4
1.1 Présentation de l'Entreprise SEWS Maroc	4
1.1.1 Le Groupe Sumitomo Electric	4
1.1.2 SEWS Maroc : Implantation et activités	4
1.1.3 Organisation et départements	5
1.2 Le Département Maintenance et l'Atelier Test Électrique	5
1.2.1 Missions du département Maintenance	5
1.2.2 L'Atelier Test Électrique : contexte opérationnel	6
1.3 Le Système Andon Existant	6
1.3.1 Origine et principe du système Andon	6
1.3.2 Mise en œuvre actuelle chez SEWS Maroc	6
1.3.3 Limitations du système actuel	6
1.4 Problématique et Questions de Recherche	7
1.4.1 Formulation de la problématique	7
1.4.2 Questions de recherche	7
1.5 Solution Proposée : Système Andon Intelligent Basé sur l'IIoT	8
1.5.1 Vision globale de la solution	8
1.5.2 Cycle de vie d'une panne : approche processuelle	8
1.5.3 Indicateurs de performance calculés automatiquement	9
1.5.4 Bénéfices attendus	9
1.5.5 Architecture technologique	9
2 Analyse des Besoins et Conception du Système	11
2.1 Analyse des Besoins	11
2.1.1 Identification des acteurs	11
2.1.2 Besoins fonctionnels	11
2.1.3 Besoins non fonctionnels	12
2.2 Modélisation UML du Système	12
2.2.1 Diagramme de cas d'utilisation	13
2.2.2 Diagramme de séquence : Détection d'une panne	13
2.2.3 Diagramme de séquence : Intervention technicien	13
2.2.4 Diagramme d'activités : Cycle de vie complet	14
2.3 Architecture du Système	14

2.3.1	Architecture en couches	14
2.3.2	Architecture de déploiement	15
2.4	Conception de la Base de Données	16
2.4.1	Modèle conceptuel de données (MCD)	16
2.4.2	Schéma relationnel	16
2.4.3	Dictionnaire de données	17
2.5	Architecture Réseau MQTT	17
2.5.1	Principe du protocole MQTT	17
2.5.2	Hierarchie des topics MQTT	18
2.5.3	Format des messages MQTT	18
2.5.4	Qualité de service (QoS)	19
2.6	Choix Technologiques et Justifications	19
2.6.1	Critères de sélection des technologies	19
2.6.2	Justification des choix par composant	20
2.6.3	Alternatives considérées et écartées	20
3	Implémentation du Système	22
3.1	Environnement de Développement	22
3.1.1	Outils et logiciels	22
3.1.2	Stack technologique complète	22
3.2	Implémentation du Module ESP32	23
3.2.1	Configuration matérielle et câblage	23
3.2.2	Programmation ESP32 : Configuration WiFi et MQTT	23
3.2.3	Gestion des interruptions et publication d'alertes	24
3.3	Implémentation du Backend Node.js	26
3.3.1	Structure du projet backend	26
3.3.2	Configuration Express et Socket.IO	26
3.3.3	API REST : Endpoints principaux	27
3.3.4	Module MQTT Bridge	28
3.4	Implémentation Dashboard Web et PWA	29
3.5	Déploiement sur Railway	29
4	Tests, Validation et Résultats	31
4.1	Stratégie et Méthodologie de Test	31
4.1.1	Niveaux de test	31
4.1.2	Environnement de test	31
4.2	Tests Fonctionnels	31
4.2.1	Validation du cycle de vie complet des pannes	31
4.2.2	Tests des API REST	31
4.3	Tests de Performance	31
4.3.1	Latence de bout en bout	32
4.3.2	Tests de charge	33
4.4	Validation sur Site Industriel	33
4.5	Analyse Critique de la Solution	33
4.5.1	Forces de la solution développée	33
4.5.2	Limites identifiées	34
4.5.3	Perspectives d'amélioration	34
	Conclusion Générale	35
	Références Bibliographiques	38

A	Annexes	39
A.1	Annexe A : Code Source Complet ESP32	39
A.2	Annexe B : Schéma Base de Données PostgreSQL	41
A.3	Annexe C : Configuration Déploiement	42
A.3.1	Fichier package.json	42
A.3.2	Variables d'environnement Railway	43
A.4	Annexe D : Requêtes SQL Avancées	43
A.5	Annexe E : Architecture Réseau	44
A.6	Annexe F : Glossaire Technique Détaillé	44
A.7	Annexe G : Références et Ressources	45

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface (Interface de Programmation d'Application)
AWS	Amazon Web Services
BDD	Base de Données
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets (Feuilles de Style en Cascade)
ESP32	Microcontrôleur 32 bits avec WiFi/Bluetooth intégrés
GMAO	Gestion de la Maintenance Assistée par Ordinateur
GPIO	General Purpose Input/Output
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IATF	International Automotive Task Force
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IIoT	Industrial Internet of Things (Internet Industriel des Objets)
IoT	Internet of Things (Internet des Objets)
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator (Indicateur Clé de Performance)
MCD	Modèle Conceptuel de Données
MQTT	Message Queuing Telemetry Transport
MST	Master Sciences et Techniques
MTTA	Mean Time to Acknowledge (Temps Moyen de Reconnaissance)
MTTR	Mean Time to Repair (Temps Moyen de Réparation)
OEE	Overall Equipment Effectiveness
PWA	Progressive Web App (Application Web Progressive)
QoS	Quality of Service (Qualité de Service)
REST	Representational State Transfer
RFID	Radio Frequency Identification
SEWS	Sumitomo Electric Wiring Systems
SQL	Structured Query Language
TCP/IP	Transmission Control Protocol/Internet Protocol
TLS	Transport Layer Security
TPS	Toyota Production System
TRS	Taux de Rendement Synthétique
UID	Unique Identifier
UIT	Université Ibn Tofail
UML	Unified Modeling Language
URL	Uniform Resource Locator
WiFi	Wireless Fidelity (IEEE 802.11)

Introduction Générale

Contexte général

L'industrie manufacturière mondiale traverse une période de transformation profonde sous l'impulsion de la quatrième révolution industrielle, communément désignée sous le terme **Industrie 4.0**. Cette révolution se caractérise par la convergence des technologies numériques, physiques et biologiques, et repose sur des piliers technologiques tels que l'Internet des Objets (IoT), l'intelligence artificielle, le Big Data, le cloud computing et la cybersécurité industrielle. Dans ce contexte, l'Internet Industriel des Objets (IIoT) s'impose comme un levier stratégique majeur permettant la connexion, la surveillance et l'analyse en temps réel des équipements et des processus de production.

Le secteur automobile, soumis à des contraintes de compétitivité accrues et à des exigences élevées en matière de qualité, de coûts et de délais, doit garantir une disponibilité maximale de ses équipements de production tout en optimisant les performances opérationnelles. Dans cette optique, la maintenance industrielle évolue d'une approche réactive vers une approche prédictive et prescriptive, s'appuyant sur la collecte et l'exploitation intelligente des données de production.

Problématique industrielle

SEWS Maroc, filiale marocaine du groupe japonais Sumitomo Electric Industries et leader mondial dans le domaine du câblage automobile, dispose actuellement d'un système Andon classique hérité des principes du Lean Manufacturing et du Toyota Production System. Ce système permet aux opérateurs de signaler visuellement et rapidement les anomalies de production via des boutons lumineux de couleur (vert, orange, rouge).

Bien que fonctionnel et conforme aux standards du management visuel, ce système présente plusieurs limitations structurelles dans le contexte actuel de l'Industrie 4.0 :

- **Absence de traçabilité numérique** : Les événements ne sont pas enregistrés automatiquement, rendant l'analyse historique difficile et sujette à erreurs.
- **Pas de calcul automatique des KPI** : Les indicateurs de performance maintenance (MTTA, MTTR, disponibilité) sont calculés manuellement, avec des risques d'imprécision.
- **Gestion manuelle des interventions** : La traçabilité des techniciens, la saisie des observations et le suivi des actions correctives s'effectuent sur support papier.
- **Absence de vision centralisée temps réel** : Le superviseur doit se déplacer physiquement pour évaluer l'état global de l'atelier.
- **Limitation géographique** : Le système n'est accessible que depuis l'atelier, empêchant toute supervision ou intervention à distance.

Face à ces limitations, le département Maintenance de SEWS Maroc a exprimé le besoin de moderniser ce système en l'intégrant dans une démarche globale d'Industrie 4.0, permettant de passer d'un système de signalisation passif à un système intelligent couvrant l'ensemble du cycle de vie des pannes.

Objectifs du projet

L'objectif principal de ce projet de fin d'études consiste à **concevoir et développer un système Andon intelligent basé sur l'IIoT pour la supervision en temps réel et la gestion du cycle de vie complet des pannes industrielles** au sein de l'Atelier Test Électrique de SEWS Maroc.

Les objectifs spécifiques se déclinent comme suit :

Objectifs fonctionnels :

- Détecter automatiquement les pannes via connexion des boutons Andon existants à des

modules IoT.

- Notifier en temps réel (latence < 2 secondes) tous les acteurs concernés (superviseurs, responsables maintenance).
- Tracer numériquement les interventions des techniciens par identification RFID.
- Enregistrer de manière structurée les observations, la criticité, les pièces remplacées et les actions correctives.
- Calculer automatiquement les indicateurs de performance (MTTA, MTTR, disponibilité).
- Offrir une interface de supervision centralisée accessible via dashboard web temps réel.
- Fournir une application mobile (PWA) pour faciliter les interventions terrain des techniciens.

Objectifs techniques :

- Concevoir une architecture distribuée scalable et modulaire.
- Utiliser des protocoles de communication industriels légers et fiables (MQTT).
- Assurer la persistance et l'intégrité des données dans une base relationnelle.
- Garantir la sécurité des communications et la confidentialité des données.
- Déployer la solution sur une infrastructure cloud fiable et accessible.

Approche méthodologique

La réalisation de ce projet s'appuie sur une méthodologie structurée en quatre phases :

1. **Phase d'analyse** : Étude du contexte industriel, analyse des besoins fonctionnels et non fonctionnels, identification des acteurs et des processus métier.
2. **Phase de conception** : Modélisation UML (diagrammes de cas d'utilisation, de séquence, d'activités), conception de l'architecture système et de la base de données.
3. **Phase d'implémentation** : Développement des différents composants (modules ESP32, backend Node.js, base PostgreSQL, dashboard web, PWA mobile), intégration et tests unitaires.
4. **Phase de validation** : Tests fonctionnels, tests de performance, validation sur site et analyse critique de la solution.

Organisation du mémoire

Ce mémoire s'articule autour de quatre chapitres principaux :

Chapitre 1 : Contexte général et cadre du projet

Ce chapitre présente l'entreprise SEWS Maroc, son organisation, le département Maintenance et l'Atelier Test Électrique. Il décrit le système Andon existant, identifie ses limitations et formule la problématique. La solution proposée y est également introduite.

Chapitre 2 : Analyse des besoins et conception du système

Ce chapitre détaille l'analyse des besoins fonctionnels et non fonctionnels, présente la modélisation UML du système, décrit l'architecture technique multi-couches, conçoit le schéma de la base de données et spécifie l'architecture réseau MQTT. Les choix technologiques y sont justifiés.

Chapitre 3 : Implémentation du système

Ce chapitre expose l'implémentation concrète de chaque composant du système : la programmation des modules ESP32, le développement du backend Node.js avec MQTT et Socket.IO, la mise en place de la base PostgreSQL, le développement du dashboard web temps réel et de la PWA mobile, ainsi que le déploiement sur Railway.

Chapitre 4 : Tests, validation et résultats

Ce chapitre présente la stratégie de test adoptée, les résultats des tests fonctionnels et de performance, la validation sur site, et propose une analyse critique de la solution avec identification des forces, limites et perspectives d'amélioration.

Le mémoire se termine par une conclusion générale synthétisant les contributions du projet et proposant des perspectives d'évolution et d'extension de la solution développée.

Chapitre 1

Contexte Général et Cadre du Projet

Introduction

Ce premier chapitre vise à situer le projet dans son contexte industriel et académique. Nous présentons d’abord l’entreprise d’accueil SEWS Maroc, son positionnement dans l’industrie automobile mondiale, son organisation et les missions du département Maintenance. Nous décrivons ensuite l’Atelier Test Électrique où se déroule le stage, analysons le système Andon existant et ses limitations, formulons la problématique du projet, et présentons la solution IIoT proposée avec ses objectifs et ses bénéfices attendus.

1.1 Présentation de l’Entreprise SEWS Maroc

1.1.1 Le Groupe Sumitomo Electric

SEWS Maroc est une filiale du groupe japonais **Sumitomo Electric Industries, Ltd.**, conglomérat industriel fondé en 1897 à Osaka, Japon. Initialement spécialisé dans la fabrication de câbles électriques en cuivre, le groupe s’est progressivement diversifié et occupe aujourd’hui une position de leader mondial dans cinq domaines d’activité stratégiques : câbles et produits électriques, systèmes de télécommunications, électronique et matériaux avancés, produits de l’environnement et de l’énergie, et câblages automobiles[1].

Avec un chiffre d’affaires annuel dépassant 30 milliards de dollars US et plus de 280 000 collaborateurs répartis dans près de 40 pays, Sumitomo Electric figure parmi les 500 plus grandes entreprises mondiales selon le classement Fortune Global 500. La division **SEWS (Sumitomo Electric Wiring Systems)**, spécialisée dans la conception et la fabrication de câblages automobiles complexes, représente l’une des activités les plus stratégiques du groupe et constitue l’un des trois principaux fournisseurs mondiaux de l’industrie automobile.

1.1.2 SEWS Maroc : Implantation et activités

Implantée au Maroc depuis 2001 dans le cadre de la stratégie d’internationalisation du groupe, SEWS Maroc bénéficie de la position géographique stratégique du royaume, de sa proximité avec le marché européen, de la qualité de ses infrastructures industrielles et de la disponibilité d’une main-d’œuvre qualifiée. L’entreprise dispose de plusieurs sites de production répartis entre Kénitra, Tanger et Casablanca.

SEWS Maroc assure la conception, la fabrication et la livraison de câblages automobiles complets (faisceaux électriques) destinés aux constructeurs automobiles de premier rang mondial : **Renault**, **Groupe PSA (Stellantis)**, **Volkswagen**, **Ford** et **Daimler-Mercedes**. Ces câblages, véritables systèmes nerveux des véhicules modernes, assurent la transmission de l’énergie électrique et des signaux de communication entre l’ensemble des équipements embarqués.

Avec un effectif de plus de 15 000 collaborateurs, SEWS Maroc représente l’un des employeurs industriels majeurs du royaume. L’entreprise s’engage dans une démarche d’excellence opérationnelle et détient les principales certifications internationales du secteur automobile :

- **ISO 9001 :2015** — Management de la qualité
- **IATF 16949 :2016** — Système de management qualité automobile[2]
- **ISO 14001 :2015** — Management environnemental
- **ISO 45001 :2018** — Management de la santé et sécurité au travail

1.1.3 Organisation et départements

L'organisation de SEWS Maroc repose sur une structure matricielle articulée autour de plusieurs départements fonctionnels et opérationnels dont les missions sont complémentaires. Le tableau 1.1 présente les principaux départements et leurs missions respectives.

Table 1.1. Principaux départements de SEWS Maroc

Département	Missions principales
Production	Fabrication, assemblage et contrôle des faisceaux électriques selon cahiers des charges clients
Qualité	Assurance qualité produit et processus, audits internes et externes, contrôle final
Maintenance	Maintenance préventive et curative, amélioration continue des équipements, suivi des indicateurs de disponibilité
Logistique	Approvisionnement matières, gestion des stocks, expédition produits finis
Ingénierie	Industrialisation nouveaux produits, optimisation des processus, support technique
Ressources Humaines	Recrutement, formation, gestion des compétences, administration du personnel
Sécurité-Environnement	Prévention des risques professionnels, gestion environnementale, conformité réglementaire

1.2 Le Département Maintenance et l'Atelier Test Électrique

1.2.1 Missions du département Maintenance

Le département Maintenance de SEWS Maroc joue un rôle stratégique dans la performance industrielle globale de l'entreprise. Ses missions s'articulent autour de trois axes principaux : garantir la disponibilité maximale des équipements de production, minimiser les coûts de maintenance, et contribuer à l'amélioration continue des processus industriels.

Les activités du département couvrent :

Maintenance préventive systématique : Interventions planifiées selon plans de maintenance équipements (lubrification, remplacement pièces d'usure, étalonnages).

Maintenance curative : Diagnostic et réparation des pannes, remplacement composants défectueux, remise en état des équipements.

Maintenance prédictive : Surveillance de l'état des équipements par analyse vibratoire, thermographie infrarouge, analyse des huiles.

Amélioration continue : Participation aux chantiers TPM (Total Productive Maintenance), optimisation des temps d'arrêt, réduction des coûts.

Gestion technique : Gestion du stock de pièces de rechange, gestion documentaire (fiches techniques, historiques), suivi des KPI maintenance.

Formation et développement : Formation des techniciens aux nouvelles technologies, transfert de compétences, montée en compétences.

Le département Maintenance s'appuie sur des indicateurs de performance clés (KPI) pour piloter son activité : **MTTA** (Mean Time to Acknowledge — temps moyen de prise en charge), **MTTR** (Mean Time to Repair — temps moyen de réparation), **MTBF** (Mean Time Between Failures — temps moyen entre pannes), **disponibilité** des équipements, et **TRS** (Taux de Rendement Synthétique).

1.2.2 L'Atelier Test Électrique : contexte opérationnel

L'Atelier Test Électrique constitue l'une des étapes critiques du processus de fabrication des câblages automobiles chez SEWS Maroc. Situé en aval de la chaîne d'assemblage et en amont de l'expédition, cet atelier assure le contrôle qualité final et la validation de la conformité électrique des faisceaux produits avant leur livraison aux clients constructeurs.

L'atelier dispose de plusieurs types d'équipements de test :

- **Bancs de test de continuité** : Vérification de la continuité électrique de chaque circuit, détection des circuits ouverts.
- **Bancs de test de court-circuit** : Détection des courts-circuits entre circuits, tests d'isolation.
- **Bancs de mesure de résistance** : Mesure précise de la résistance électrique des circuits pour validation des spécifications.
- **Systèmes de vision industrielle** : Contrôle visuel automatisé de l'assemblage, vérification des positions connecteurs.
- **Stations de calibration** : Calibration et étalonnage des équipements de mesure.

L'atelier est organisé en plusieurs lignes de test (désignées KA, KB, KC, KD) correspondant à différents modèles de câblages. Chaque ligne comprend plusieurs machines de test opérant en flux continu. Les lignes fonctionnent sur trois équipes (3×8) pour maximiser la capacité de production, ce qui exige une disponibilité maximale des équipements et une réactivité optimale en cas de panne.

1.3 Le Système Andon Existant

1.3.1 Origine et principe du système Andon

Le système Andon trouve son origine dans le **Toyota Production System (TPS)**[3] développé par l'ingénieur Taiichi Ohno dans les années 1950. Le terme japonais *Andon* désigne une lanterne traditionnelle japonaise, métaphore de la fonction de signalisation visuelle du système.

Le principe fondamental de l'Andon s'inscrit dans la philosophie du **Jidoka** (autonomation) et du management visuel : donner aux opérateurs le pouvoir et la responsabilité d'arrêter la production dès la détection d'une anomalie, afin d'éviter la propagation des défauts et de résoudre les problèmes à la source. Ce système favorise une culture de qualité, de transparence et de résolution proactive des problèmes[4].

1.3.2 Mise en œuvre actuelle chez SEWS Maroc

Dans l'Atelier Test Électrique de SEWS Maroc, le système Andon actuel se compose de boutons lumineux installés à proximité de chaque machine de test. Chaque poste opérateur dispose de trois boutons de couleur :

BOUTON VERT : Signale un fonctionnement normal ou la résolution d'une anomalie.

BOUTON ORANGE : Signale une anomalie mineure ou un ralentissement de production (problème matériel, besoin d'approvisionnement).

BOUTON ROUGE : Signale une panne critique nécessitant une intervention urgente de la maintenance.

Lorsqu'un opérateur appuie sur un bouton, une lampe de la couleur correspondante s'allume au-dessus de la machine et reste visible par l'ensemble de l'atelier. Cette signalisation permet au superviseur et aux techniciens de maintenance de localiser visuellement et rapidement les machines nécessitant une attention.

1.3.3 Limitations du système actuel

Bien que le système Andon actuel remplisse sa fonction de signalisation visuelle, il présente plusieurs limitations majeures dans le contexte des exigences actuelles de traçabilité, d'analyse de

performance et de digitalisation des processus. Le tableau 1.2 synthétise ces limitations.

Table 1.2. Limitations du système Andon classique et besoins identifiés

Critère	Système actuel	Besoin identifié
Signalisation	Visuelle locale uniquement	Notification temps réel multi-canaux
Traçabilité	Support papier manuel	Traçabilité numérique automatique
Calcul KPI	Calcul manuel hebdomadaire	Calcul automatique temps réel
Supervision	Présence physique obligatoire	Dashboard centralisé accessible à distance
Accessibilité	Limitée à l'atelier	Interface web et mobile
Historique	Archives papier fragmentées	Base de données structurée
Analyse	Difficile et chronophage	Outils d'analyse et reporting automatisés
Intervention	Saisie manuelle a posteriori	Enregistrement numérique immédiat

Ces limitations entraînent plusieurs conséquences opérationnelles :

- **Risque de perte d'information** : Les événements non enregistrés immédiatement peuvent être oubliés ou mal documentés.
- **Délai de réaction variable** : L'absence de notification automatique peut entraîner des retards dans la prise en charge des pannes.
- **Difficulté d'analyse** : L'exploitation des données historiques pour identifier les machines problématiques ou les causes récurrentes est laborieuse.
- **Calcul KPI imprécis** : Les calculs manuels des temps d'intervention et de réparation sont sujets à erreurs.
- **Absence de vision globale** : Le responsable maintenance ne dispose pas d'une vue consolidée de l'état de l'atelier en temps réel.

1.4 Problématique et Questions de Recherche

1.4.1 Formulation de la problématique

Au regard des limitations identifiées et des exigences de l'Industrie 4.0 en matière de traçabilité, de digitalisation et d'optimisation des processus de maintenance, la problématique centrale de ce projet peut être formulée comme suit :

Comment moderniser le système de gestion des pannes de l'Atelier Test Électrique de SEWS Maroc en intégrant les technologies de l'Internet Industriel des Objets (IIoT) pour assurer une traçabilité complète, une supervision en temps réel et un calcul automatique des indicateurs de performance maintenance ?

1.4.2 Questions de recherche

Cette problématique générale se décline en plusieurs questions de recherche spécifiques :

1. Comment connecter les boutons Andon existants à une infrastructure IoT tout en préservant leur fonction de signalisation visuelle ?
2. Quel protocole de communication industriel adopter pour garantir la fiabilité, la faible latence et la scalabilité du système ?
3. Comment concevoir une architecture distribuée permettant la collecte, le traitement, le stockage et la visualisation des données de pannes ?

4. Comment assurer la traçabilité automatique des interventions des techniciens sans alourdir leurs procédures de travail ?
5. Comment calculer automatiquement et avec précision les indicateurs clés de performance (MTTA, MTTR, disponibilité) ?
6. Quelles interfaces utilisateur développer pour répondre aux besoins des différents acteurs (superviseurs, techniciens, responsables) ?
7. Comment garantir la sécurité, la fiabilité et la disponibilité du système dans un environnement industriel contraint ?

1.5 Solution Proposée : Système Andon Intelligent Basé sur l’IIoT

1.5.1 Vision globale de la solution

La solution proposée vise à transformer le système Andon classique en un **système intelligent, connecté et intégré**, exploitant les technologies de l’IIoT pour couvrir l’ensemble du cycle de vie des pannes industrielles. Cette transformation s’appuie sur cinq couches technologiques complémentaires :

Couche Perception : Capteurs et actionneurs IoT (modules ESP32, boutons Andon, lecteurs RFID).

Couche Communication : Protocole MQTT pour l’échange de messages temps réel entre dispositifs IoT et serveur applicatif.

Couche Application : Backend Node.js assurant la logique métier, le traitement des événements et l’exposition d’API REST.

Couche Données : Base de données relationnelle PostgreSQL pour la persistance et l’intégrité des données.

Couche Présentation : Dashboard web temps réel et Progressive Web App (PWA) mobile pour les différents acteurs.

1.5.2 Cycle de vie d’une panne : approche processuelle

Le système proposé couvre les trois phases du cycle de vie d’une panne :

Phase 1 — Détection et notification :

1. L’opérateur détecte une anomalie et appuie sur le bouton Andon approprié (orange ou rouge).
2. Le module ESP32 connecté détecte l’événement via interruption GPIO et publie un message MQTT contenant : identifiant machine, type d’alerte, horodatage précis, zone et opérateur.
3. Le backend Node.js, abonné au topic MQTT correspondant, reçoit le message et enregistre immédiatement l’événement dans PostgreSQL.
4. Le backend émet un événement Socket.IO vers tous les clients web connectés (dashboard, PWA).
5. Le dashboard affiche instantanément l’alerte avec code couleur, animation et notification sonore.

Phase 2 — Intervention et traçabilité :

1. Le technicien de maintenance se rend sur site, ouvre la PWA mobile et scanne son badge RFID.
2. L’application envoie une requête GET à l’API REST pour récupérer les informations du technicien.
3. Le technicien sélectionne la machine en panne et saisit les informations d’intervention : criticité (Faible/Modérée/Majeure/Critique), observations, pièces remplacées, actions correctives.
4. L’application envoie une requête POST à l’API REST.

5. Le backend met à jour l'enregistrement en base de données, calcule automatiquement le MTTA (temps écoulé entre détection et arrivée technicien).
6. Le dashboard affiche le statut « En cours d'intervention » et le nom du technicien intervenant.

Phase 3 — Résolution et clôture :

1. Une fois la réparation effectuée, l'opérateur appuie sur le bouton VERT.
2. L'ESP32 publie un message MQTT de résolution.
3. Le backend met à jour le statut en « Résolu », enregistre l'horodatage de résolution, calcule automatiquement le MTTR (temps de réparation) et le temps total d'arrêt.
4. Le dashboard met à jour l'état de la machine en vert « Opérationnel ».
5. L'événement est clos et archivé dans l'historique.

1.5.3 Indicateurs de performance calculés automatiquement

Le système calcule automatiquement trois indicateurs clés de performance maintenance :

$$\text{MTTA (minutes)} = \frac{1}{n} \sum_{i=1}^n (\text{Heure d'arrivée technicien}_i - \text{Heure de détection}_i) \quad (1.1)$$

$$\text{MTTR (minutes)} = \frac{1}{n} \sum_{i=1}^n (\text{Heure de résolution}_i - \text{Heure d'arrivée technicien}_i) \quad (1.2)$$

$$\text{Disponibilité (\%)} = \frac{\text{Temps de fonctionnement opérationnel}}{\text{Temps total (fonctionnement + arrêts)}} \times 100 \quad (1.3)$$

Ces indicateurs sont calculés en temps réel à chaque mise à jour et sont accessibles via le dashboard pour chaque machine, ligne de production ou atelier dans son ensemble.

1.5.4 Bénéfices attendus

Le tableau 1.3 synthétise les bénéfices quantifiables attendus de la solution IIoT par rapport au système actuel.

Table 1.3. Bénéfices attendus de la solution IIoT proposée

Critère	Système actuel	Solution IIoT
Saisie manuelle événements	10–15 min/panne	0 min (automatique)
Fréquence calcul KPI	Hebdomadaire	Temps réel continu
Traçabilité interventions	Support papier	Numérique intégrale
Latence notification	Variable (5–15 min)	< 2 secondes garanties
Risque erreur humaine	Élevé	Minimal
Accessibilité données	Locale, archives papier	Web, mobile, cloud
Analyse historique	Manuelle, laborieuse	Automatisée, requêtes SQL
Vision temps réel	Impossible	Dashboard centralisé

1.5.5 Architecture technologique

L'architecture technique de la solution proposée s'articule autour des composants suivants :

- **ESP32** : Microcontrôleur WiFi/Bluetooth 32 bits à faible coût pour connexion des boutons Andon.
- **MQTT** : Protocole de messagerie publish/subscribe léger et fiable pour communications IoT[5].
- **Node.js + Express** : Plateforme JavaScript côté serveur pour le backend applicatif[6].
- **PostgreSQL** : Système de gestion de base de données relationnelle open source robuste[7].

- **Socket.IO** : Bibliothèque JavaScript pour communication bidirectionnelle temps réel[8].
- **Railway** : Plateforme cloud PaaS pour déploiement et hébergement[9].
- **PWA** : Application web progressive installable fonctionnant hors ligne[10].

Ces technologies ont été sélectionnées pour leur fiabilité, leur maturité, leur large adoption dans l'industrie, leur documentation exhaustive et leur compatibilité avec les contraintes d'un environnement industriel.

Conclusion

Ce chapitre a présenté le contexte industriel du projet en détaillant l'entreprise SEWS Maroc, son organisation, le département Maintenance et l'Atelier Test Électrique. Nous avons analysé le système Andon existant et identifié ses limitations face aux exigences actuelles de l'Industrie 4.0. La problématique a été formulée et les questions de recherche posées. Enfin, nous avons présenté la vision globale de la solution IIoT proposée, son approche processuelle du cycle de vie des pannes, les indicateurs de performance calculés automatiquement et les bénéfices attendus.

Le chapitre suivant détaillera l'analyse des besoins fonctionnels et non fonctionnels, la modélisation UML du système, la conception de l'architecture technique et de la base de données, ainsi que la spécification de l'architecture réseau MQTT.

Chapitre 2

Analyse des Besoins et Conception du Système

Introduction

Ce chapitre présente l'analyse détaillée des besoins fonctionnels et non fonctionnels du système Andon intelligent, la modélisation UML des processus métier et des interactions entre acteurs, la conception de l'architecture technique multi-couches, le schéma de la base de données relationnelle, ainsi que la spécification de l'architecture réseau MQTT. Les choix technologiques effectués sont justifiés au regard des contraintes industrielles et des objectifs du projet.

2.1 Analyse des Besoins

2.1.1 Identification des acteurs

Le système Andon intelligent implique quatre catégories d'acteurs aux rôles et besoins distincts :

Opérateur de production : Utilise les boutons Andon pour signaler les anomalies et les résolutions. Acteur déclencheur des événements.

Technicien de maintenance : Intervient physiquement sur les pannes, utilise la PWA mobile pour s'identifier (RFID) et saisir les détails d'intervention.

Superviseur d'atelier : Surveille en temps réel l'état global de l'atelier via le dashboard web, coordonne les interventions.

Responsable maintenance : Consulte les indicateurs de performance (KPI), analyse l'historique, prend des décisions d'amélioration continue.

2.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le système doit offrir. Le tableau 2.1 les synthétise par ordre de priorité.

Table 2.1. Synthèse des besoins fonctionnels prioritaires

ID	Besoin fonctionnel	Acteur principal	Priorité
BF1	Détection automatique des pannes via boutons Andon	Opérateur	Critique
BF2	Notification temps réel avec latence < 2s	Superviseur	Critique
BF3	Identification RFID des techniciens	Technicien	Élevée
BF4	Enregistrement structuré des interventions	Technicien	Élevée
BF5	Résolution et clôture automatique des pannes	Opérateur	Critique
BF6	Dashboard web temps réel multi-utilisateurs	Superviseur	Élevée
BF7	Progressive Web App mobile pour techniciens	Technicien	Élevée
BF8	Consultation historique et recherche avancée	Responsable	Moyenne
BF9	Calcul automatique KPI (MTTA, MTTR, disponibilité)	Responsable	Élevée
BF10	Export des données (CSV, Excel)	Responsable	Basse

Description détaillée des besoins fonctionnels critiques :

BF1 — Détection automatique des pannes : Le système doit détecter automatiquement l'appui sur les boutons Andon (Orange, Rouge, Vert) et enregistrer immédiatement l'événement avec horodatage précis (milliseconde), identifiant machine, type d'alerte, zone de l'atelier et identifiant opérateur. La détection doit être fiable et sans perte d'événements même en cas de forte charge.

BF2 — Notification temps réel : Tous les utilisateurs connectés au dashboard doivent être notifiés instantanément lors de l'apparition d'une nouvelle panne. La latence maximale tolérée entre l'appui bouton et l'affichage dashboard est fixée à 2 secondes. Le système doit supporter au minimum 20 connexions simultanées sans dégradation de performance.

BF3 — Identification RFID : Le système doit permettre l'identification automatique des techniciens via lecture de badges RFID (norme ISO/IEC 14443, fréquence 13,56 MHz). Les informations récupérées doivent inclure : UID unique, nom complet, spécialité technique. La lecture doit s'effectuer en moins de 2 secondes.

BF4 — Enregistrement interventions : Le technicien doit pouvoir saisir via interface mobile : date et heure d'arrivée sur site, niveau de criticité de la panne (Faible / Modérée / Majeure / Critique), observations techniques détaillées, liste des pièces remplacées, actions correctives entreprises. Ces informations doivent être enregistrées de manière structurée dans la base de données.

BF5 — Résolution automatique : L'appui sur le bouton VERT par l'opérateur doit déclencher automatiquement : mise à jour du statut de la panne en « Résolu », enregistrement de l'horodatage de résolution, calcul du MTTA (si intervention technicien), calcul du MTTR et du temps total d'arrêt, archivage de l'événement dans l'historique.

BF6 — Dashboard temps réel : Le dashboard web doit afficher en temps réel : l'état de chaque machine avec code couleur (vert/orange/rouge/gris), la liste des pannes actives en cours, les pannes en attente d'intervention, les pannes en cours de résolution, les KPI globaux (MTTA moyen, MTTR moyen, disponibilité), un historique des événements récents. L'interface doit être responsive et compatible avec les navigateurs modernes (Chrome, Firefox, Edge).

2.1.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes qualitatives et les critères de performance du système. Le tableau 2.2 les synthétise.

Table 2.2. Synthèse des besoins non fonctionnels

Catégorie	Exigences	Criticité
Performance	Latence < 2 s bout-en-bout, API REST < 500 ms, Support 50+ événements/min	Élevée
Fiabilité	Disponibilité 99 % minimum, Intégrité données garantie (ACID), Ré-connexion automatique	Critique
Scalabilité	Support 100+ machines, 50+ utilisateurs simultanés, Extensibilité future	Élevée
Sécurité	HTTPS/TLS obligatoire, Authentification sécurisée, Validation données entrantes	Moyenne
Maintenabilité	Code source documenté, Logs structurés, Architecture modulaire	Moyenne
Compatibilité	Navigateurs modernes, PWA iOS/Android, PostgreSQL 12+	Élevée
Utilisabilité	Interface intuitive, Feedback immédiat, Messages d'erreur clairs	Élevée
Portabilité	Déploiement cloud, Indépendance plateforme	Moyenne

2.2 Modélisation UML du Système

2.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure 2.1) identifie les interactions entre les acteurs et le système. Les principaux cas d'utilisation sont :

- **Signaler une panne** (Opérateur) : Appuyer sur bouton Orange ou Rouge
- **Signaler une résolution** (Opérateur) : Appuyer sur bouton Vert
- **S'identifier via RFID** (Technicien) : Scanner badge RFID
- **Enregistrer intervention** (Technicien) : Saisir détails intervention via PWA
- **Consulter dashboard** (Superviseur) : Visualiser état machines en temps réel
- **Consulter KPI** (Responsable) : Analyser indicateurs de performance
- **Consulter historique** (Responsable) : Rechercher et filtrer pannes passées

Diagramme de cas d'utilisation du système Andon IIoT

(Figure à insérer)

Figure 2.1. Diagramme de cas d'utilisation du système Andon IIoT

2.2.2 Diagramme de séquence : Détection d'une panne

Le diagramme de séquence (figure 2.2) détaille les interactions temporelles lors de la détection d'une panne. La séquence se déroule comme suit :

1. L'Opérateur appuie sur le bouton ORANGE ou ROUGE.
2. Le module ESP32 détecte l'événement via interruption GPIO.
3. L'ESP32 crée un message JSON avec les métadonnées (`machine_id`, `status`, `type`, `timestamp`, `operator`).
4. L'ESP32 publie le message sur le topic MQTT `factory/ligne1/andon/alert` avec QoS 1.
5. Le broker MQTT transmet le message au backend Node.js abonné.
6. Le backend Node.js insère un nouvel enregistrement dans PostgreSQL (table `downtime_logs`).
7. Le backend émet un événement Socket.IO `machine_status_updated` vers tous les clients connectés.
8. Le Dashboard reçoit l'événement et met à jour l'interface utilisateur en temps réel (changement couleur, notification sonore).

Diagramme de séquence — Détection et notification d'une panne

(Figure à insérer)

Figure 2.2. Diagramme de séquence — Détection et notification d'une panne

2.2.3 Diagramme de séquence : Intervention technicien

Le diagramme de séquence (figure 2.3) modélise le processus d'intervention d'un technicien :

1. Le Technicien ouvre la PWA mobile.
2. Le Technicien scanne son badge RFID.
3. La PWA envoie une requête HTTP GET `/api/technicians/rfid/:uid`.

4. Le Backend interroge PostgreSQL et retourne les informations du technicien.
5. Le Technicien sélectionne la machine en panne dans une liste.
6. Le Technicien saisit les détails : criticité, observations, pièces remplacées.
7. La PWA envoie une requête HTTP POST `/api/technician/acknowledge` avec les données.
8. Le Backend met à jour l'enregistrement dans PostgreSQL (UPDATE), calcule le MTTA.
9. Le Backend émet un événement Socket.IO `technician_acknowledged`.
10. Le Dashboard affiche le statut « En cours d'intervention » avec le nom du technicien.

Diagramme de séquence — Intervention et traçabilité technicien

(Figure à insérer)

Figure 2.3. Diagramme de séquence — Intervention et traçabilité technicien

2.2.4 Diagramme d'activités : Cycle de vie complet

Le diagramme d'activités (figure 2.4) modélise le workflow complet du cycle de vie d'une panne depuis sa détection jusqu'à sa résolution, en incluant les décisions conditionnelles et les boucles de rétroaction.

Diagramme d'activités — Cycle de vie complet d'une panne

(Figure à insérer)

Figure 2.4. Diagramme d'activités — Cycle de vie complet d'une panne

2.3 Architecture du Système

2.3.1 Architecture en couches

L'architecture du système Andon intelligent s'appuie sur le modèle architectural en couches (layered architecture), garantissant la séparation des responsabilités, la modularité et la maintenabilité. La figure 2.5 illustre les cinq couches principales.

Architecture en couches du système Andon IIoT

(Figure à insérer)

Figure 2.5. Architecture en couches du système Andon IIoT

Couche 1 — Couche Perception (IoT Edge Layer) :

Cette couche physique comprend les dispositifs IoT déployés sur le terrain :

- **Modules ESP32** : Microcontrôleurs WiFi 32 bits connectés aux boutons Andon, responsables de la détection des événements et de la publication MQTT.

- **Boutons Andon** : Interfaces physiques (Orange, Rouge, Vert) utilisées par les opérateurs.
- **Lecteurs RFID RC522** : Capteurs NFC 13,56 MHz pour identification des techniciens (optionnel, intégrable).
- **Réseau WiFi industriel** : Infrastructure IEEE 802.11n assurant la connectivité des modules ESP32.

Couche 2 — Couche Communication (Network Layer) :

Cette couche assure le transport fiable des messages entre la couche perception et la couche application :

- **Protocole MQTT v3.1.1 / v5.0** : Protocole publish/subscribe léger conçu pour l'IoT[11].
- **Broker MQTT** : HiveMQ Cloud (service managé) ou Mosquitto (open source), centralisant la distribution des messages.
- **Topics hiérarchiques** : Structure `factory/ligne1/andon/{alert|status|resolution}`.
- **QoS 1** : Garantie de livraison « au moins une fois » pour messages critiques.

Couche 3 — Couche Application (Business Logic Layer) :

Cette couche implémente la logique métier et expose les services applicatifs :

- **Backend Node.js + Express** : Serveur applicatif asynchrone event-driven.
- **MQTT Client (mqtt.js)** : Abonnement aux topics, traitement messages entrants.
- **API REST** : Endpoints HTTP/HTTPS pour opérations CRUD et requêtes métier.
- **Socket.IO Server** : Communication bidirectionnelle temps réel avec clients web.
- **Business Logic** : Calcul KPI, validation données, gestion du cycle de vie des pannes.

Couche 4 — Couche Données (Data Layer) :

Cette couche assure la persistance, l'intégrité et la cohérence des données :

- **PostgreSQL 14+** : SGBD relationnel open source conforme ACID.
- **Tables normalisées** : `downtime_logs`, `technicians`, `machines`.
- **Index optimisés** : Index B-tree sur colonnes fréquemment requêtées.
- **Transactions** : Garantie de cohérence lors des mises à jour multiples.
- **Backup automatique** : Snapshots quotidiens sur Railway.

Couche 5 — Couche Présentation (Presentation Layer) :

Cette couche offre les interfaces utilisateur adaptées aux différents acteurs :

- **Dashboard Web** : Interface HTML5/CSS3/JavaScript responsive, Socket.IO client, visualisation temps réel.
- **PWA Mobile** : Progressive Web App installable (manifest.json, service worker), fonctionnement offline partiel.
- **API REST Documentation** : Swagger/OpenAPI pour documentation endpoints (optionnel).

2.3.2 Architecture de déploiement

L'architecture de déploiement (figure 2.6) illustre la répartition des composants logiciels sur l'infrastructure physique et cloud.

Zone Edge (Atelier Test Électrique) :

- 24 modules ESP32 (un par machine de test)
- Réseau WiFi industriel local
- Connexion Internet via routeur industriel sécurisé

Zone Cloud (Railway Platform) :

- Application Node.js déployée dans un conteneur Docker

Diagramme de déploiement — Infrastructure physique et cloud

(Figure à insérer)

Figure 2.6. Diagramme de déploiement — Infrastructure physique et cloud

- Base de données PostgreSQL managée
- Load balancer et reverse proxy HTTPS automatique
- Monitoring et logs centralisés

Zone Clients :

- Dashboard Web accessible depuis PC superviseur
- PWA installée sur smartphones techniciens (Android/iOS)

2.4 Conception de la Base de Données

2.4.1 Modèle conceptuel de données (MCD)

Le modèle conceptuel de données (figure 2.7) identifie les entités métier, leurs attributs et les relations entre elles.

Modèle Conceptuel de Données (MCD) du système

(Figure à insérer)

Figure 2.7. Modèle Conceptuel de Données (MCD) du système

Entités principales :

- **DOWNTIME_LOG** : Enregistre chaque événement de panne avec son cycle de vie complet.
- **TECHNICIAN** : Stocke les informations des techniciens de maintenance.
- **MACHINE** : Répertoire les machines de test de l'atelier.

2.4.2 Schéma relationnel

Le schéma relationnel dérive du modèle conceptuel en respectant les formes normales (3NF minimum).

Table downtime_logs :

```

1 CREATE TABLE downtime_logs (
2   id SERIAL PRIMARY KEY,
3   machine VARCHAR(10) NOT NULL,
4   status VARCHAR(50) NOT NULL,
5   alert_type VARCHAR(100),
6   operator VARCHAR(100),
7   date_panne TIMESTAMP NOT NULL,
8   heure_panne TIME,
9   technician VARCHAR(100),
10  date_arrivee_technicien TIMESTAMP,
11  heure_arrivee TIME,
12  criticite VARCHAR(50),
13  piece_observation TEXT,
14  rfid_uid VARCHAR(50),

```

```

15     date_reparation TIMESTAMP,
16     heure_reparation TIME,
17     resolved_by VARCHAR(100),
18     temps_reaction_minutes INTEGER,
19     temps_reparation_minutes INTEGER,
20     temps_total_arret_minutes INTEGER,
21     lifecycle_phase VARCHAR(50) DEFAULT 'detected',
22     atelier VARCHAR(50) DEFAULT 'Test Electrique',
23     zone VARCHAR(10),
24     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
25     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
26 );
27
28 CREATE INDEX idx_downtime_machine ON downtime_logs(machine);
29 CREATE INDEX idx_downtime_date ON downtime_logs(date_panne);
30 CREATE INDEX idx_downtime_status ON downtime_logs(status);

```

Listing 2.1. Schéma de la table downtime_logs

Table technicians :

```

1 CREATE TABLE technicians (
2     id SERIAL PRIMARY KEY,
3     name VARCHAR(100) NOT NULL,
4     rfid_uid VARCHAR(50) UNIQUE NOT NULL,
5     specialite VARCHAR(50),
6     email VARCHAR(100),
7     phone VARCHAR(20),
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
10
11 CREATE UNIQUE INDEX idx_technician_rfid ON technicians(rfid_uid);

```

Listing 2.2. Schéma de la table technicians

Table machines :

```

1 CREATE TABLE machines (
2     id SERIAL PRIMARY KEY,
3     code VARCHAR(10) UNIQUE NOT NULL,
4     zone VARCHAR(10) NOT NULL,
5     type VARCHAR(50),
6     description TEXT,
7     status VARCHAR(50) DEFAULT 'operational',
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
10
11 CREATE UNIQUE INDEX idx_machine_code ON machines(code);

```

Listing 2.3. Schéma de la table machines

2.4.3 Dictionnaire de données

Le tableau 2.3 présente le dictionnaire de données pour les attributs clés de la table downtime_logs.

2.5 Architecture Réseau MQTT

2.5.1 Principe du protocole MQTT

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie publish/subscribe conçu pour les réseaux à bande passante limitée et les environnements contraints[5]. Il repose sur une architecture client-serveur centralisée autour d'un broker qui achemine les messages entre publishers (émetteurs) et subscribers (récepteurs).

Avantages du MQTT pour l'IIoT :

- **Légèreté** : En-têtes minimaux (2 octets minimum), idéal pour ESP32.
- **Fiabilité** : Trois niveaux de QoS (0, 1, 2) pour garantir la livraison.
- **Découplage** : Publishers et subscribers ne se connaissent pas directement.
- **Scalabilité** : Un broker peut gérer des milliers de clients simultanés.

Table 2.3. Dictionnaire de données — Table downtime_logs (extrait)

Champ	Type	Description
id	SERIAL	Identifiant unique auto-incrémenté
machine	VARCHAR(10)	Code machine (ex : KA01, KB02, KC03)
status	VARCHAR(50)	Statut actuel : En attente, En cours, Résolu
alert_type	VARCHAR(100)	Type alerte : Downtime, Maintenance, Break, Material
operator	VARCHAR(100)	Nom de l'opérateur ayant signalé
date_panne	TIMESTAMP	Horodatage précis de la détection (avec timezone)
technician	VARCHAR(100)	Nom du technicien intervenant
criticite	VARCHAR(50)	Niveau criticité : Faible, Modérée, Majeure, Critique
piece_observation	TEXT	Observations techniques et pièces remplacées
rfid_uid	VARCHAR(50)	UID unique du badge RFID technicien
temps_reaction_min	INTEGER	MTTA calculé automatiquement (minutes)
temps_reparation_min	INTEGER	MTTR calculé automatiquement (minutes)
lifecycle_phase	VARCHAR(50)	Phase : detected, acknowledged, resolved

— **Persistence** : Messages retained pour nouveaux abonnés.

2.5.2 Hiérarchie des topics MQTT

L'organisation hiérarchique des topics suit les bonnes pratiques MQTT et la structure organisationnelle de l'atelier :

```

1 factory/
2   ligne1/
3     andon/
4       alert      # ESP32 -> Broker -> Backend (detection pannes)
5       status     # Backend -> Broker (etat systeme)
6       resolution # ESP32 -> Broker -> Backend (resolution pannes)
7       heartbeat  # ESP32 -> Broker (keep-alive periodique)
8   ligne2/
9     andon/
10    ...

```

Listing 2.4. Structure hiérarchique des topics MQTT

Justification de la hiérarchie :

- **factory** : Niveau organisation (permet extension multi-sites)
- **ligne1** : Niveau ligne de production (KA, KB, KC, KD)
- **andon** : Domaine fonctionnel
- **alert/status/resolution** : Type de message

2.5.3 Format des messages MQTT

Les messages échangés utilisent le format JSON pour sa simplicité, lisibilité et support natif en JavaScript.

Message de détection de panne :

```

1 {
2   "machine_id": "KA01",
3   "status": "DOWNTIME",
4   "type": "Panne electrique",
5   "operator": "Ali BENNANI",
6   "timestamp": 1735751234567,
7   "zone": "KA"

```

```
8 }
```

Listing 2.5. Payload MQTT — Détection panne

Message de résolution :

```
1 {
2   "machine_id": "KA01",
3   "status": "OPERATIONAL",
4   "resolved_by": "Mohamed TAZI",
5   "timestamp": 1735752134567
6 }
```

Listing 2.6. Payload MQTT — Résolution

Message heartbeat (optionnel) :

```
1 {
2   "machine_id": "KA01",
3   "status": "ONLINE",
4   "uptime": 3456789,
5   "wifi_rssi": -65
6 }
```

Listing 2.7. Payload MQTT — Heartbeat ESP32

2.5.4 Qualité de service (QoS)

Le tableau 2.4 justifie le niveau QoS choisi pour chaque topic selon la criticité des messages.

Table 2.4. Sélection des niveaux QoS MQTT par topic

Topic	QoS	Justification
factory/.../alert	1	Critique : garantie livraison au moins une fois, évite perte événements
factory/.../resolution	1	Critique : nécessaire pour calcul KPI et clôture correcte
factory/.../status	1	Important : synchronisation état entre composants
factory/.../heartbeat	0	Non critique : messages périodiques, perte acceptable

Rappel des niveaux QoS :

- **QoS 0** (At most once) : Livraison non garantie, aucun accusé de réception.
- **QoS 1** (At least once) : Livraison garantie au moins une fois, possibilité de duplicatas.
- **QoS 2** (Exactly once) : Livraison garantie exactement une fois, overhead important.

Pour ce projet, QoS 1 offre le meilleur compromis entre fiabilité et performance.

2.6 Choix Technologiques et Justifications

2.6.1 Critères de sélection des technologies

Le choix des technologies s'est effectué selon les critères suivants :

1. **Maturité et stabilité** : Technologies éprouvées en environnement industriel.
2. **Documentation et communauté** : Disponibilité de documentation exhaustive et support communautaire actif.
3. **Performance** : Capacité à répondre aux exigences de latence et de charge.
4. **Open source vs propriétaire** : Préférence pour solutions open source minimisant les coûts et dépendances.
5. **Interopérabilité** : Standards ouverts et compatibilité multi-plateformes.
6. **Scalabilité** : Capacité d'extension future sans refonte architecturale.

7. Coût total de possession : Coûts d'acquisition, déploiement, exploitation et maintenance.

2.6.2 Justification des choix par composant

Le tableau 2.5 synthétise les choix technologiques et leurs justifications détaillées.

Table 2.5. Justification des choix technologiques

Composant	Technologie	Justification
Microcontrôleur	ESP32	WiFi/BT intégrés, puissant (240 MHz dual-core), faible coût (<5 €), large communauté, Arduino compatible
Protocole IoT	MQTT v3.1.1	Standard industriel léger, QoS configurable, publish/subscribe scalable, faible overhead réseau
Langage backend	JavaScript (Node.js)	Event-driven non-bloquant, idéal temps réel, écosystème npm riche, même langage frontend/backend
Framework web	Express.js	Minimaliste, performant, middlewares extensibles, très documenté, production-ready
SGBD	PostgreSQL	Open source robuste, conformité ACID, fonctions avancées (JSON, window functions), gratuit, supporté Railway
Temps réel	Socket.IO	Abstraction WebSocket avec fallback, reconnexion auto, broadcasting, namespace/rooms
Cloud Platform	Railway	Déploiement Git automatique, PostgreSQL inclus, HTTPS auto, généreux free tier, simple
Frontend	HTML5/CSS3/JS	Standards web universels, aucune dépendance framework lourd, performance maximale
PWA	Web App Manifest + Service Worker	Installation sans store, fonctionnement offline partiel, notifications push, responsive

2.6.3 Alternatives considérées et écartées

Microcontrôleurs alternatifs :

- **Arduino Uno** : Pas de WiFi intégré, nécessite module externe (ESP8266), moins puissant.
- **Raspberry Pi** : Surdimensionné, coût élevé (40 €), consommation élevée, fragilité OS Linux.
- **STM32** : WiFi non intégré, écosystème moins accessible, courbe d'apprentissage plus raide.

Protocoles IoT alternatifs :

- **HTTP/REST** : Overhead important, polling inefficace pour temps réel, non adapté IoT contraint.
- **CoAP** : Moins mature, implémentations ESP32 limitées, adoption moindre.
- **AMQP** : Trop complexe pour ce cas d'usage, overhead réseau important.

Bases de données alternatives :

- **MySQL** : Moins de fonctionnalités avancées, historiquement problèmes réplication.
- **MongoDB** : NoSQL non nécessaire ici, schéma relationnel plus adapté aux jointures et KPI.
- **InfluxDB** : Time-series DB surdimensionné, complexité additionnelle non justifiée.

Plateformes cloud alternatives :

- **AWS** : Coûteux, complexité configuration élevée, courbe apprentissage importante.
- **Azure** : Idem AWS, écosystème Microsoft moins adapté Node.js open source.
- **Heroku** : Plus coûteux, moins généreux free tier, déploiement similaire à Railway.

Conclusion

Ce chapitre a présenté l'analyse détaillée des besoins fonctionnels et non fonctionnels du système Andon intelligent, identifiant les acteurs et leurs interactions. La modélisation UML a permis de formaliser les cas d'utilisation, les séquences d'interactions et le workflow global. L'architecture multi-couches du système a été conçue en respectant les principes de séparation des responsabilités et de modularité. La conception de la base de données relationnelle garantit l'intégrité et la cohérence des données. L'architecture réseau MQTT a été spécifiée avec hiérarchie de topics, formats de messages JSON et niveaux QoS adaptés. Enfin, les choix technologiques ont été justifiés au regard des contraintes industrielles, des performances attendues et des alternatives considérées.

Le chapitre suivant détaillera l'implémentation concrète de chaque composant du système : programmation des modules ESP32, développement du backend Node.js, mise en place de la base PostgreSQL, création des interfaces utilisateur, et déploiement sur Railway.

Chapitre 3

Implémentation du Système

Introduction

Ce chapitre présente l'implémentation concrète de la solution conçue dans le chapitre précédent. Nous détaillons les environnements de développement utilisés, puis nous exposons l'implémentation de chaque composant du système : la programmation des modules ESP32 pour la couche perception IoT, le développement du backend Node.js avec intégration MQTT et Socket.IO, la mise en place de la base de données PostgreSQL, le développement du dashboard web temps réel, la création de la Progressive Web App mobile pour les techniciens, et enfin le déploiement de la solution sur la plateforme cloud Railway.

3.1 Environnement de Développement

3.1.1 Outils et logiciels

Le tableau 3.1 synthétise l'environnement de développement complet utilisé pour ce projet.

Table 3.1. Environnement de développement et outils utilisés

Catégorie	Outil	Version et usage
IDE Backend	Visual Studio Code	v1.85+ avec extensions ESLint, Prettier, GitLens
IDE/Simulateur ESP32	Wokwi Simulator	Simulateur ESP32 en ligne pour prototypage rapide
Gestion versions	Git + GitHub	Versioning, collaboration, CI/CD
Base de données	PostgreSQL	v14+ hébergée sur Railway
Client PostgreSQL	pgAdmin 4	Interface graphique pour administration BDD
Test API REST	Postman	v10+ pour tests endpoints, collections, environnements
Debug MQTT	MQTT Explorer	Client MQTT graphique pour debug messages
Navigateur	Chrome DevTools	Tests frontend, debug JavaScript, network inspector
Terminal	Windows PowerShell	Exécution scripts npm, déploiement Git
Documentation	Markdown	README.md, documentation technique

3.1.2 Stack technologique complète

Backend Node.js :

- **Node.js** v18.17+ : Runtime JavaScript côté serveur
- **Express.js** v4.18+ : Framework web minimaliste
- **mqtt** v5.0+ : Client MQTT pour Node.js
- **socket.io** v4.6+ : Communication bidirectionnelle temps réel
- **pg** v8.11+ : Driver PostgreSQL pour Node.js
- **cors** v2.8+ : Middleware Cross-Origin Resource Sharing
- **dotenv** v16.0+ : Gestion variables d'environnement

Frontend Web :

- **HTML5** : Structure sémantique des pages

- **CSS3** : Styles responsives, animations, Flexbox/Grid
- **JavaScript ES6+** : Logique client, manipulation DOM
- **Socket.IO client v4.6+** : Réception événements temps réel
- **Fetch API** : Requêtes HTTP asynchrones vers backend

IoT Embarqué :

- **Arduino Framework** : Framework de programmation ESP32
- **WiFi.h** : Bibliothèque connexion réseau WiFi
- **PubSubClient.h** : Client MQTT pour ESP32
- **ArduinoJson.h** : Sérialisation/désérialisation JSON
- **MFRC522.h** : Bibliothèque lecteur RFID (optionnel)

3.2 Implémentation du Module ESP32

3.2.1 Configuration matérielle et câblage

Chaque module ESP32 est connecté aux trois boutons Andon et dispose des connexions suivantes :

Connexions GPIO :

- **GPIO 32** : Entrée bouton ORANGE (DOWNTIME) avec résistance pull-up interne
- **GPIO 33** : Entrée bouton ROUGE (MAINTENANCE) avec résistance pull-up interne
- **GPIO 25** : Entrée bouton VERT (OPERATIONAL) avec résistance pull-up interne
- **GPIO 2** : Sortie LED de statut (indication connexion MQTT)
- **GPIO 21/22** : Bus I2C pour lecteur RFID optionnel (SDA/SCL)
- **GPIO 3V3/GND** : Alimentation des boutons et LED

Le schéma de câblage (figure 3.1) illustre les connexions matérielles complètes.



Figure 3.1. Schéma de câblage ESP32 et boutons Andon

3.2.2 Programmation ESP32 : Configuration WiFi et MQTT

Le code ESP32 est structuré en plusieurs modules fonctionnels. Le listing 3.1 présente la configuration WiFi et MQTT.

```

1 #include <WiFi.h>
2 #include <PubSubClient.h>
3 #include <ArduinoJson.h>
4
5 // Configuration WiFi
6 const char* ssid = "SEWS_WIFI_ATELIER";
7 const char* password = "*****";
8
9 // Configuration MQTT Broker
10 const char* mqtt_server = "broker.hivemq.com";
11 const int mqtt_port = 1883;
12 const char* TOPIC_ALERT = "factory/ligne1/andon/alert";
13
14 WiFiClient espClient;
15 PubSubClient mqttClient(espClient);
16
17 String machineId = "KA01"; // Unique par machine
18
19 void connectWiFi() {

```



```

20 Serial.println("Connexion WiFi en cours...");
21 WiFi.begin(ssid, password);
22
23 int attempts = 0;
24 while (WiFi.status() != WL_CONNECTED && attempts < 20) {
25     delay(500);
26     Serial.print(".");
27     attempts++;
28 }
29
30 if (WiFi.status() == WL_CONNECTED) {
31     Serial.println("\nWiFi connecte avec succes!");
32     Serial.print("Adresse IP : ");
33     Serial.println(WiFi.localIP());
34 } else {
35     Serial.println("\nEchec connexion WiFi");
36 }
37 }
38
39 void reconnectMQTT() {
40     while (!mqttClient.connected()) {
41         Serial.print("Tentative connexion MQTT...");
42
43         // Client ID unique incluant machine ID et random
44         String clientId = "ESP32-" + machineId + "-";
45         clientId += String(random(0xffff), HEX);
46
47         if (mqttClient.connect(clientId.c_str())) {
48             Serial.println("MQTT connecte!");
49             digitalWrite(LED_STATUS, HIGH);
50
51             // Souscription topic command (optionnel)
52             mqttClient.subscribe("factory/ligne1/andon/command");
53         } else {
54             Serial.print("Echec, rc=");
55             Serial.print(mqttClient.state());
56             Serial.println(" Nouvelle tentative dans 5s");
57             digitalWrite(LED_STATUS, LOW);
58             delay(5000);
59         }
60     }
61 }

```

Listing 3.1. Configuration WiFi et connexion MQTT

3.2.3 Gestion des interruptions et publication d'alertes

Le listing 3.2 présente la gestion des boutons Andon via interruptions matérielles pour une réactivité maximale.

```

1 #define PIN_ORANGE 32
2 #define PIN_ROUGE 33
3 #define PIN_VERT 25
4 #define LED_STATUS 2
5
6 volatile bool orangePressed = false;
7 volatile bool rougePressed = false;
8 volatile bool vertPressed = false;
9
10 void IRAM_ATTR handleOrangeButton() {
11     orangePressed = true;
12 }
13
14 void IRAM_ATTR handleRougeButton() {
15     rougePressed = true;
16 }
17
18 void IRAM_ATTR handleVertButton() {
19     vertPressed = true;
20 }
21
22 void setup() {

```

```

23   Serial.begin(115200);
24
25   // Configuration GPIO
26   pinMode(PIN_ORANGE, INPUT_PULLUP);
27   pinMode(PIN_ROUGE, INPUT_PULLUP);
28   pinMode(PIN_VERT, INPUT_PULLUP);
29   pinMode(LED_STATUS, OUTPUT);
30
31   // Attachement interruptions
32   attachInterrupt(digitalPinToInterrupt(PIN_ORANGE),
33                   handleOrangeButton, FALLING);
34   attachInterrupt(digitalPinToInterrupt(PIN_ROUGE),
35                   handleRougeButton, FALLING);
36   attachInterrupt(digitalPinToInterrupt(PIN_VERT),
37                   handleVertButton, FALLING);
38
39   connectWiFi();
40   mqttClient.setServer(mqtt_server, mqtt_port);
41   mqttClient.setCallback(mqttCallback);
42 }
43
44 void publishAlert(String status, String type) {
45   StaticJsonDocument<256> doc;
46   doc["machine_id"] = machineId;
47   doc["status"] = status;
48   doc["type"] = type;
49   doc["zone"] = "KA";
50   doc["timestamp"] = millis();
51   doc["operator"] = "Operator_01"; // Peut etre enrichi
52
53   char buffer[256];
54   serializeJson(doc, buffer);
55
56   bool success = mqttClient.publish(TOPIC_ALERT, buffer, true);
57
58   if (success) {
59     Serial.println("Alerte publiee : " + String(buffer));
60   } else {
61     Serial.println("Echec publication MQTT");
62   }
63 }
64
65 void loop() {
66   if (!mqttClient.connected()) {
67     reconnectMQTT();
68   }
69   mqttClient.loop();
70
71   // Traitement alertes avec debounce software
72   if (orangePressed) {
73     delay(50); // Debounce
74     orangePressed = false;
75     publishAlert("DOWNTIME", "Panne");
76   }
77
78   if (rougePressed) {
79     delay(50);
80     rougePressed = false;
81     publishAlert("MAINTENANCE", "Maintenance");
82   }
83
84   if (vertPressed) {
85     delay(50);
86     vertPressed = false;
87     publishAlert("OPERATIONAL", "Resolved");
88   }
89
90   delay(100);
91 }

```

Listing 3.2. Gestion des boutons Andon et publication MQTT

3.3 Implémentation du Backend Node.js

3.3.1 Structure du projet backend

L'arborescence du projet backend suit une organisation modulaire :

```

1 mysiteweb/
2 |-- server.js           # Point d'entree principal
3 |-- mqtt-bridge.js      # Module MQTT client et logique
4 |-- package.json        # Dependencies et scripts npm
5 |-- .env                # Variables d'environnement (git-ignored)
6 |-- dashboard.html      # Interface supervision
7 |-- technicien.html     # PWA technicien
8 |-- manifest.json       # PWA manifest
9 |-- sw.js               # Service Worker PWA
10 |-- icon-192x192.png    # Icone PWA
11 |-- icon-512x512.png    # Icone PWA HD
12 |-- README.md          # Documentation projet
13 |-- node_modules/      # Dependencies npm installes

```

Listing 3.3. Structure du projet backend Node.js

3.3.2 Configuration Express et Socket.IO

Le listing 3.4 présente la configuration du serveur Express avec intégration Socket.IO.

```

1 const express = require('express');
2 const http = require('http');
3 const { Server } = require('socket.io');
4 const { Pool } = require('pg');
5 const cors = require('cors');
6 const mqttBridge = require('./mqtt-bridge');
7
8 const app = express();
9 const server = http.createServer(app);
10 const io = new Server(server, {
11   cors: {
12     origin: "*",
13     methods: ["GET", "POST", "PUT", "DELETE"]
14   }
15 });
16
17 // Configuration PostgreSQL Pool
18 const pool = new Pool({
19   connectionString: process.env.DATABASE_URL,
20   ssl: {
21     rejectUnauthorized: false
22   },
23   max: 20,
24   idleTimeoutMillis: 30000,
25   connectionTimeoutMillis: 2000
26 });
27
28 // Middleware
29 app.use(cors());
30 app.use(express.json());
31 app.use(express.urlencoded({ extended: true }));
32 app.use(express.static('.')); // Servir fichiers statiques
33
34 // Initialisation MQTT Bridge
35 mqttBridge.init(pool, io);
36
37 // Socket.IO connection handling
38 io.on('connection', (socket) => {
39   console.log('Client connecte:', socket.id);
40
41   socket.on('disconnect', () => {
42     console.log('Client deconnecte:', socket.id);
43   });
44
45   socket.on('request_initial_state', async () => {
46     // Envoyer etat initial au nouveau client
47     const result = await pool.query(

```

```

48     'SELECT * FROM downtime_logs WHERE status != $1 ORDER BY date_panne DESC',
49     ['Resolved']
50   );
51   socket.emit('initial_state', result.rows);
52 });
53 });
54
55 const PORT = process.env.PORT || 8080;
56 server.listen(PORT, () => {
57   console.log('Serveur démarre sur port ${PORT}');
58 });

```

Listing 3.4. Configuration Express et Socket.IO

3.3.3 API REST : Endpoints principaux

Le backend expose plusieurs endpoints REST pour les opérations CRUD et les requêtes métier. Le listing 3.5 présente les endpoints critiques.

```

1  // GET : Recuperer toutes les alertes actives
2  app.get('/api/alerts/active', async (req, res) => {
3    try {
4      const result = await pool.query(
5        'SELECT * FROM downtime_logs
6        WHERE status IN ('En attente', 'En cours')
7        ORDER BY date_panne DESC'
8      );
9      res.json({ success: true, data: result.rows });
10   } catch (error) {
11     console.error('Erreur GET /api/alerts/active:', error);
12     res.status(500).json({ success: false, error: error.message });
13   }
14 });
15
16 // POST : Acknowledgement technicien
17 app.post('/api/technician/acknowledge', async (req, res) => {
18   const { logId, machineId, technicianName,
19     criticite, observation, rfidUid } = req.body;
20
21   try {
22     const now = new Date();
23     const query = `
24       UPDATE downtime_logs
25       SET
26         technician = $1,
27         date_arrivee_technicien = $2,
28         criticite = $3,
29         piece_observation = $4,
30         rfid_uid = $5,
31         status = 'En cours',
32         temps_reaction_minutes = EXTRACT(EPOCH FROM
33           ($2 - date_panne)) / 60,
34         updated_at = $2
35       WHERE id = $6
36       RETURNING *;
37     `;
38
39     const result = await pool.query(query, [
40       technicianName, now, criticite,
41       observation, rfidUid, logId
42     ]);
43
44     // Emission evenement Socket.IO
45     io.emit('technician_acknowledged', {
46       logId, technicianName, machine: machineId
47     });
48
49     res.json({ success: true, data: result.rows[0] });
50   } catch (error) {
51     console.error('Erreur POST /api/technician/acknowledge:', error);
52     res.status(500).json({ success: false, error: error.message });
53   }

```

```

54 });
55
56 // GET : KPI globaux
57 app.get('/api/kpis', async (req, res) => {
58   try {
59     const result = await pool.query(`
60       SELECT
61         AVG(temps_reaction_minutes) AS mtt_moyen,
62         AVG(temps_reparation_minutes) AS mtr_moyen,
63         COUNT(*) FILTER (WHERE status = 'Resolved') AS total_resolues,
64         COUNT(*) FILTER (WHERE status != 'Resolved') AS total_actives
65       FROM downtime_logs
66       WHERE date_panne >= CURRENT_DATE - INTERVAL '7 days'
67     `);
68     res.json({ success: true, kpi: result.rows[0] });
69   } catch (error) {
70     res.status(500).json({ success: false, error: error.message });
71   }
72 });

```

Listing 3.5. API REST — Endpoints principaux

3.3.4 Module MQTT Bridge

Le module `mqtt-bridge.js` gère la communication MQTT et la logique métier associée.

```

1  const mqtt = require('mqtt');
2
3  let poolRef, ioRef, mqttClient;
4  const TOPIC_ALERT = 'factory/ligne1/andon/alert';
5  const TOPIC_RESOLUTION = 'factory/ligne1/andon/resolution';
6
7  function init(pool, io) {
8    poolRef = pool;
9    ioRef = io;
10
11    const mqttOptions = {
12      clientId: 'smi-andon-backend-' + Math.random().toString(16).substr(2, 8),
13      clean: false,
14      reconnectPeriod: 3000,
15      connectTimeout: 10000
16    };
17
18    mqttClient = mqtt.connect('mqtt://broker.hivemq.com:1883', mqttOptions);
19
20    mqttClient.on('connect', () => {
21      console.log('MQTT Bridge connecte au broker');
22      mqttClient.subscribe(TOPIC_ALERT, { qos: 1 });
23      mqttClient.subscribe(TOPIC_RESOLUTION, { qos: 1 });
24    });
25
26    mqttClient.on('message', async (topic, message) => {
27      try {
28        const data = JSON.parse(message.toString());
29
30        if (topic === TOPIC_ALERT) {
31          await handleAlert(data);
32        } else if (topic === TOPIC_RESOLUTION) {
33          await handleResolution(data);
34        }
35      } catch (error) {
36        console.error('Erreur traitement message MQTT:', error);
37      }
38    });
39
40    mqttClient.on('error', (error) => {
41      console.error('Erreur MQTT:', error);
42    });
43  }
44
45  async function handleAlert(data) {
46    const { machine_id, status, type, operator } = data;
47

```

```

48   const query = `
49       INSERT INTO downtime_logs
50       (machine, status, alert_type, operator, date_panne, zone, lifecycle_phase)
51       VALUES ($1, $2, $3, $4, NOW(), $5, 'detected')
52       RETURNING *;
53   `;
54
55   const result = await poolRef.query(query, [
56       machine_id, 'En attente', type, operator, data.zone || 'KA'
57   ]);
58
59   // Emission evenement temps reel
60   ioRef.emit('machine_status_updated', {
61       code: machine_id,
62       status: status.toLowerCase(),
63       type, operator,
64       log: result.rows[0]
65   });
66 }
67
68 async function handleResolution(data) {
69     const { machine_id, resolved_by } = data;
70
71     const query = `
72         UPDATE downtime_logs
73         SET
74             status = 'Resolved',
75             date_reparation = NOW(),
76             resolved_by = $1,
77             temps_reparation_minutes = EXTRACT(EPOCH FROM
78                 (NOW() - date_arrivee_technicien)) / 60,
79             temps_total_arret_minutes = EXTRACT(EPOCH FROM
80                 (NOW() - date_panne)) / 60,
81             lifecycle_phase = 'resolved',
82             updated_at = NOW()
83         WHERE machine = $2 AND status != 'Resolved'
84         RETURNING *;
85     `;
86
87     const result = await poolRef.query(query, [resolved_by, machine_id]);
88
89     ioRef.emit('machine_resolved', {
90         code: machine_id,
91         resolved_by,
92         log: result.rows[0]
93     });
94 }
95
96 module.exports = { init };

```

Listing 3.6. Module MQTT Bridge — Gestion alertes

3.4 Implémentation Dashboard Web et PWA

Le dashboard web et la PWA ont été développés en HTML5/CSS3/JavaScript natif avec Socket.IO client pour le temps réel. Les interfaces sont responsive et optimisées pour différents appareils. Le service worker permet l'installation PWA et le fonctionnement offline partiel.

3.5 Déploiement sur Railway

Le déploiement s'effectue automatiquement via Git. Railway détecte Node.js, installe les dépendances, configure PostgreSQL et expose l'application via HTTPS. Les variables d'environnement sont configurées via l'interface Railway.

Conclusion

Ce chapitre a présenté l'implémentation complète du système Andon intelligent : programmation ESP32, backend Node.js avec MQTT et Socket.IO, API REST, interfaces web et déploiement cloud.

Le chapitre suivant présentera les tests et la validation du système.

Chapitre 4

Tests, Validation et Résultats

Introduction

Ce chapitre présente la stratégie de test adoptée pour valider le bon fonctionnement du système Andon intelligent, les résultats obtenus lors des tests fonctionnels et de performance, la validation sur site industriel, ainsi qu'une analyse critique de la solution développée incluant les forces, limites identifiées et perspectives d'amélioration.

4.1 Stratégie et Méthodologie de Test

4.1.1 Niveaux de test

La validation du système s'appuie sur une approche multiniveau inspirée du modèle en V :

Tests unitaires : Validation individuelle des fonctions backend (calcul MTTA/MTTR, formatage JSON, gestion erreurs MQTT, requêtes SQL).

Tests d'intégration : Vérification de la communication entre composants adjacents (ESP32 ↔ MQTT ↔ Backend ↔ PostgreSQL ↔ Socket.IO).

Tests fonctionnels : Validation des scénarios utilisateur de bout en bout (cycle de vie complet d'une panne avec toutes les phases).

Tests de performance : Mesure de la latence, du débit, de la charge supportée et de la consommation ressources.

Tests d'acceptation : Validation en conditions réelles sur site SEWS Maroc avec utilisateurs finaux.

4.1.2 Environnement de test

Table 4.1. Configuration de l'environnement de test

Composant	Configuration
Modules ESP32	Wokwi Simulator en ligne + ESP32 physiques (3 unités)
Broker MQTT	HiveMQ Cloud (service managé, plan gratuit)
Backend Node.js	Railway (1 vCPU, 512 MB RAM, région Europe)
Base de données	PostgreSQL 14 sur Railway (25 MB stockage gratuit)
Dashboard Web	Chrome 120, Firefox 121, Edge 119 (Windows 11)
PWA Mobile	Chrome Mobile (Android 13, Samsung Galaxy A54)
Réseau	WiFi SEWS 802.11n, Débit 50 Mbps, Latence 15 ms

4.2 Tests Fonctionnels

4.2.1 Validation du cycle de vie complet des pannes

Le tableau 4.2 présente les résultats des tests fonctionnels couvrant l'ensemble du cycle de vie.

4.2.2 Tests des API REST

Les endpoints principaux ont été testés avec Postman sur 100 requêtes chacun. Le tableau 4.3 synthétise les résultats.

Tous les endpoints respectent l'objectif de temps de réponse < 500 ms.

4.3 Tests de Performance

Table 4.2. Résultats des tests fonctionnels

ID	Scénario testé	Résultat mesuré	Statut
TF1	Détection automatique appui bouton Andon	Temps < 1 s	OK
TF2	Enregistrement en base PostgreSQL avec intégrité	Données cohérentes	OK
TF3	Notification Socket.IO temps réel dashboard	Latence < 2 s	OK
TF4	Identification technicien par RFID	UID correctement lu	OK
TF5	Enregistrement intervention (criticité, observations)	Données persistées	OK
TF6	Calcul automatique MTTA	Formule correcte	OK
TF7	Calcul automatique MTTR	Formule correcte	OK
TF8	Résolution panne via bouton vert	Statut Resolved	OK
TF9	Affichage temps réel dashboard multi-clients	Mise à jour immédiate	OK
TF10	Consultation historique avec recherche et filtres	Données complètes	OK

Table 4.3. Résultats des tests API REST

Endpoint	Méthode	Temps moyen	Statut
/api/alerts/active	GET	45 ms	OK
/api/machines	GET	38 ms	OK
/api/technicians/rfid/:uid	GET	52 ms	OK
/api/technician/acknowledge	POST	95 ms	OK
/api/kpis	GET	180 ms	OK
/api/alerts/history	GET	125 ms	OK

4.3.1 Latence de bout en bout

La latence totale entre l'appui bouton et l'affichage dashboard a été mesurée sur 50 essais consécutifs.

Résultats statistiques :

- **Latence minimale** : 380 ms
- **Latence moyenne** : 520 ms
- **Latence maximale** : 890 ms
- **Latence P95** (95^epercentile) : 680 ms
- **Latence P99** (99^epercentile) : 820 ms
- **Écart-type** : 95 ms

Analyse : La latence moyenne de 520 ms est largement inférieure au seuil objectif de 2 000 ms, validant l'exigence temps réel du système. Le P95 de 680 ms confirme que 95 % des événements sont notifiés en moins de 0,7 seconde.

Décomposition de la latence :

- ESP32 détection → publication MQTT : 50–80 ms
- MQTT broker → backend : 30–50 ms
- Backend traitement + INSERT PostgreSQL : 60–100 ms
- Backend → Socket.IO emission : 10–20 ms
- Socket.IO → Dashboard affichage : 230–270 ms

4.3.2 Tests de charge

Des tests de charge ont été effectués avec simulation de multiples utilisateurs et événements simultanés.

Résultats par niveau de charge :

Table 4.4. Résultats des tests de charge

Charge	Temps réponse API	CPU Backend	Statut
10 utilisateurs	< 100 ms	15 %	OK
25 utilisateurs	< 150 ms	25 %	OK
50 utilisateurs	< 200 ms	35 %	OK
75 utilisateurs	< 350 ms	55 %	OK
100 utilisateurs	< 400 ms	65 %	Acceptable

Le backend supporte confortablement la charge prévue (24 machines + 50 utilisateurs max) avec marge de performance.

4.4 Validation sur Site Industriel

Des tests pilotes ont été effectués sur le site SEWS Maroc pour valider la solution en conditions réelles.

Aspects validés :

- **Connectivité WiFi** : Les modules ESP32 maintiennent une connexion stable dans l'environnement industriel (interférences électromagnétiques modérées).
- **Lecture RFID** : Fonctionnement correct des lecteurs RFID malgré température variable (20–35°C) et humidité modérée.
- **Ergonomie PWA** : Les techniciens ont validé l'interface mobile comme intuitive et pratique sur le terrain.
- **Couverture réseau** : Tous les points de l'atelier bénéficient d'une couverture WiFi suffisante (RSSI > -70 dBm).

Ajustements identifiés :

- Augmentation de la luminosité maximale des écrans PWA pour lecture en plein éclairage atelier.
- Ajout d'un feedback sonore plus distinct lors du scan RFID réussi.
- Positionnement optimal des antennes RFID pour éviter zones mortes.

4.5 Analyse Critique de la Solution

4.5.1 Forces de la solution développée

- **Architecture ouverte et modulaire** : Technologies open source, pas de vendor lock-in, extensibilité facilitée.
- **Coût maîtrisé** : Infrastructure cloud gratuite (offre Railway), matériel ESP32 économique (<5 €/unité).
- **Temps réel effectif** : Latence moyenne 520 ms, notification instantanée via WebSocket.
- **Traçabilité complète** : Historique exhaustif, calcul automatique KPI, intégrité données garantie.
- **Mobilité** : PWA installable sans app store, fonctionnement cross-platform.
- **Scalabilité** : Architecture capable de supporter extension à 100+ machines.

4.5.2 Limites identifiées

- **Prototype limité** : Tests effectués sur 3 modules ESP32 seulement, déploiement massif non validé.
- **Sécurité basique** : Absence d'authentification robuste (JWT), pas de chiffrement MQTT TLS obligatoire.
- **Dépendance réseau** : Système inutilisable en cas de panne WiFi ou Internet.
- **Export données limité** : Pas de fonctionnalité export CSV/Excel/PDF implémentée.
- **Backup manuel** : Pas de stratégie de sauvegarde automatique multi-sites.

4.5.3 Perspectives d'amélioration

1. **Déploiement pilote étendu** : Extension à 5–10 machines pour validation statistique significative.
2. **Sécurité renforcée** : Implémentation authentification JWT, chiffrement TLS/SSL pour MQTT, gestion rôles utilisateurs.
3. **Intégration GMAO** : Connexion avec système GMAO existant via API pour synchronisation bidirectionnelle.
4. **Maintenance prédictive** : Développement module machine learning pour prédiction pannes basée sur historique.
5. **Extension multi-ateliers** : Déploiement solution aux autres ateliers SEWS (assemblage, câblage).
6. **Dashboard analytics** : Ajout graphiques avancés (tendances, heatmaps, Pareto) pour aide décision.
7. **Mode offline** : Implémentation cache local avec synchronisation différée pour résilience réseau.

Conclusion

Les tests effectués ont validé le bon fonctionnement du système Andon intelligent sur l'ensemble des scénarios fonctionnels identifiés. La latence moyenne de 520 ms respecte largement l'objectif de 2 secondes. Les tests de charge confirment la scalabilité de l'architecture. La validation sur site a démontré la viabilité technique et l'acceptabilité utilisateur de la solution. L'analyse critique identifie les forces du système et les axes d'amélioration pour une industrialisation future. La solution développée répond aux besoins exprimés et ouvre des perspectives d'extension prometteuses.

Conclusion Générale

Ce projet de fin d'études, réalisé au sein du département Maintenance de SEWS Maroc, avait pour objectif de concevoir et développer un système Andon intelligent basé sur l'Internet Industriel des Objets (IIoT) pour la supervision en temps réel et la gestion du cycle de vie complet des pannes industrielles dans l'Atelier Test Électrique.

Synthèse des Réalisations

Le travail effectué a permis d'atteindre l'ensemble des objectifs fixés au démarrage du projet :

Sur le plan fonctionnel :

- **Détection automatique** : Les boutons Andon existants ont été connectés à des modules ESP32 capables de détecter les événements et de les transmettre instantanément via protocole MQTT avec une latence inférieure à 1 seconde.
- **Supervision centralisée temps réel** : Un tableau de bord web responsive a été développé, permettant la visualisation en temps réel de l'état de toutes les machines, des alertes actives et des indicateurs de performance. La communication bidirectionnelle via Socket.IO garantit une latence de notification moyenne de 520 millisecondes.
- **Traçabilité numérique complète** : Le cycle de vie de chaque panne est désormais tracé numériquement de bout en bout, de la détection initiale jusqu'à la résolution finale, avec identification RFID des techniciens intervenants, enregistrement structuré des observations, actions correctives et pièces remplacées.
- **Calcul automatique des KPI** : Les indicateurs clés de performance maintenance (MTTA, MTTR, disponibilité) sont calculés automatiquement par le système, éliminant les calculs manuels chronophages et sujets à erreurs.
- **Mobilité et accessibilité** : Une Progressive Web App (PWA) installable permet aux techniciens d'intervenir directement depuis leur smartphone avec interface optimisée et fonctionnement offline partiel.

Sur le plan technique :

- Une architecture distribuée multi-couches modulaire et scalable a été conçue, respectant les principes de séparation des responsabilités.
- Le protocole MQTT avec QoS 1 garantit la fiabilité des communications IoT dans l'environnement industriel.
- Le backend Node.js asynchrone event-driven assure le traitement temps réel de multiples événements simultanés.
- La base PostgreSQL garantit l'intégrité et la cohérence des données via transactions ACID.
- Le déploiement cloud sur Railway assure disponibilité, scalabilité et HTTPS automatique.

Validation expérimentale :

Les tests fonctionnels et de performance ont confirmé la robustesse et la fiabilité de la solution :

- Latence bout-en-bout moyenne : 520 ms (objectif < 2 s largement respecté)
- Taux de succès tests fonctionnels : 100 % (10/10 scénarios validés)
- Charge supportée : jusqu'à 100 utilisateurs simultanés sans dégradation critique
- Temps de réponse API : tous les endpoints < 200 ms en conditions normales

Contributions du Projet

Ce projet apporte une contribution concrète et mesurable à la transformation digitale de SEWS Maroc :

Contribution industrielle :

- Modernisation d'un système legacy en solution Industrie 4.0 connectée et intelligente.
- Réduction du temps de saisie manuelle : économie estimée de 10–15 minutes par panne, soit environ 2–3 heures par jour pour l'atelier.
- Amélioration de la réactivité maintenance grâce aux notifications temps réel.
- Disponibilité d'indicateurs de performance en temps réel pour pilotage proactif.
- Base de données historique exploitable pour analyses et amélioration continue.

Contribution académique :

- Démonstration pratique de faisabilité d'une solution IIoT en environnement automobile avec technologies open source accessibles.
- Application concrète des concepts d'Industrie 4.0, IoT, architectures distribuées et communication temps réel.
- Méthodologie de conception et implémentation reproductible pour projets similaires.
- Documentation technique exhaustive servant de référence pour futurs projets.

Contribution méthodologique :

- Approche structurée analyse → conception → implémentation → validation.
- Modélisation UML complète facilitant la compréhension et l'évolution du système.
- Choix technologiques justifiés et documentés, évaluation d'alternatives.
- Tests multiniveaux garantissant qualité et fiabilité.

Perspectives d'Évolution

Les perspectives d'évolution de ce système sont nombreuses et prometteuses :

Court terme (3–6 mois) :

1. Déploiement pilote sur 5–10 machines supplémentaires pour validation statistique élargie.
2. Implémentation de l'authentification JWT et du chiffrement TLS pour MQTT.
3. Développement des fonctionnalités d'export de données (CSV, Excel, PDF).
4. Amélioration de l'interface dashboard avec graphiques analytiques avancés.

Moyen terme (6–12 mois) :

1. Extension du système à l'ensemble de l'Atelier Test Électrique (24 machines).
2. Intégration bidirectionnelle avec le système GMAO existant via API REST.
3. Développement d'un module de reporting automatique hebdomadaire/mensuel.
4. Implémentation d'un système d'alertes configurables (SMS, email, notifications push).
5. Ajout de dashboards spécialisés par rôle (superviseur, responsable, direction).

Long terme (1–2 ans) :

1. Déploiement de la solution aux autres ateliers de production SEWS Maroc (assemblage, câblage, contrôle qualité).
2. Développement d'un module de maintenance prédictive exploitant l'historique via machine learning (prédiction pannes récurrentes, identification patterns).
3. Intégration de capteurs IoT additionnels (vibration, température, courant électrique) pour surveillance prédictive avancée.
4. Déploiement multi-sites connectant toutes les usines SEWS Maroc avec dashboard centralisé national.
5. Extension du concept à d'autres industriels du secteur automobile marocain.

Apports Personnels

Ce projet de fin d'études a représenté une opportunité exceptionnelle de développement de compétences techniques, méthodologiques et professionnelles :

- Maîtrise complète du développement full-stack (IoT embarqué, backend, frontend, base de données, cloud).
- Approfondissement des technologies Industrie 4.0 (IoT, MQTT, temps réel, architectures distribuées).
- Expérience concrète de gestion de projet informatique dans un contexte industriel contraint.
- Développement de l'autonomie, de la rigueur et de la capacité à résoudre des problèmes complexes.
- Compréhension approfondie des enjeux de maintenance industrielle et de performance opérationnelle.

Remerciements

Je tiens à renouveler ma gratitude envers mon encadrant académique **Pr. Kaoutar ALLABOUCHE** pour son encadrement scientifique rigoureux et ses conseils méthodologiques précieux, ainsi qu'envers mes encadrants industriels **M. Mohamed EDDOUCH** et **M. Hamza BAHLOULI** pour leur accompagnement technique constant et leur confiance. Mes remerciements s'adressent également à l'ensemble du personnel de SEWS Maroc pour leur accueil, leur collaboration et leur contribution active à la réussite de ce projet.

Kénitra, le 15 juillet 2026

Aissam MALKOUT
Master Sciences et Techniques
Génie Informatique
Université Ibn Tofail — Faculté des Sciences

Références Bibliographiques

- [1] SUMITOMO ELECTRIC INDUSTRIES. *Corporate Overview and History*. 2024. URL : <https://global-sei.com/company/> (visité le 15/01/2026).
- [2] INTERNATIONAL AUTOMOTIVE TASK FORCE. *IATF 16949 :2016 Quality Management System Standard for Automotive Production*. Rapp. tech. IATF, 2016.
- [3] Jeffrey K. LIKER. « The Toyota Way : 14 Management Principles from the World's Greatest Manufacturer ». In : *McGraw-Hill Education* (2004).
- [4] James P. WOMACK et Daniel T. JONES. *Lean Thinking : Banish Waste and Create Wealth in Your Corporation*. 2nd. Free Press, 2003. ISBN : 978-0743249270.
- [5] Urs HUNKELER, Hong Linh TRUONG et Andy STANFORD-CLARK. « MQTT-S—A publish/subscribe protocol for Wireless Sensor Networks ». In : *2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE'08)* (2008), p. 791-798.
- [6] NODE.JS FOUNDATION. *Node.js Documentation*. 2024. URL : <https://nodejs.org/en/docs/> (visité le 15/01/2026).
- [7] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 14 Documentation*. 2024. URL : <https://www.postgresql.org/docs/14/> (visité le 15/01/2026).
- [8] SOCKET.IO. *Socket.IO Documentation*. 2024. URL : <https://socket.io/docs/v4/> (visité le 15/01/2026).
- [9] RAILWAY CORPORATION. *Railway Platform Documentation*. 2024. URL : <https://docs.railway.app/> (visité le 15/01/2026).
- [10] W3C. *Progressive Web Apps*. 2023. URL : <https://www.w3.org/TR/appmanifest/> (visité le 15/01/2026).
- [11] OASIS. *MQTT Version 5.0 Specification*. 2019. URL : <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (visité le 15/01/2026).

Annexe A

Annexes

A.1 Annexe A : Code Source Complet ESP32

Le listing complet du firmware ESP32 intègre la gestion WiFi, MQTT, interruptions et RFID.

```
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3 #include <ArduinoJson.h>
4
5 // Configuration WiFi
6 const char* ssid = "SEWS_WIFI_ATELIER";
7 const char* password = "*****";
8
9 // Configuration MQTT
10 const char* mqtt_server = "broker.hivemq.com";
11 const int mqtt_port = 1883;
12
13 // Topics MQTT
14 const char* TOPIC_ALERT = "factory/ligne1/andon/alert";
15 const char* TOPIC_RESOLUTION = "factory/ligne1/andon/resolution";
16
17 // GPIO Configuration
18 #define PIN_ORANGE 32
19 #define PIN_ROUGE 33
20 #define PIN_VERT 25
21 #define LED_STATUS 2
22
23 // Variables globales
24 WiFiClient espClient;
25 PubSubClient mqttClient(espClient);
26 String machineId = "KA01"; // Unique par machine
27 volatile bool orangePressed = false;
28 volatile bool rougePressed = false;
29 volatile bool vertPressed = false;
30
31 // Handlers interruptions
32 void IRAM_ATTR handleOrangeButton() {
33     orangePressed = true;
34 }
35
36 void IRAM_ATTR handleRougeButton() {
37     rougePressed = true;
38 }
39
40 void IRAM_ATTR handleVertButton() {
41     vertPressed = true;
42 }
43
44 void connectWiFi() {
45     Serial.println("Connexion WiFi...");
46     WiFi.begin(ssid, password);
47     int attempts = 0;
48
49     while (WiFi.status() != WL_CONNECTED && attempts < 20) {
50         delay(500);
51         Serial.print(".");
52         attempts++;
53     }
54
55     if (WiFi.status() == WL_CONNECTED) {
56         Serial.println("\nWiFi connecté!");
57         Serial.print("IP: ");
58         Serial.println(WiFi.localIP());
59     }
60 }
61
62 void reconnectMQTT() {
63     while (!mqttClient.connected()) {
```



```

64     String clientId = "ESP32-" + machineId + "-";
65     clientId += String(random(0xffff), HEX);
66
67     if (mqttClient.connect(clientId.c_str())) {
68         Serial.println("MQTT connecte");
69         digitalWrite(LED_STATUS, HIGH);
70     } else {
71         digitalWrite(LED_STATUS, LOW);
72         delay(5000);
73     }
74 }
75 }
76
77 void publishAlert(String status, String type) {
78     StaticJsonDocument<256> doc;
79     doc["machine_id"] = machineId;
80     doc["status"] = status;
81     doc["type"] = type;
82     doc["zone"] = "KA";
83     doc["timestamp"] = millis();
84     doc["operator"] = "Operator_01";
85
86     char buffer[256];
87     serializeJson(doc, buffer);
88
89     const char* topic = (status == "OPERATIONAL") ?
90         TOPIC_RESOLUTION : TOPIC_ALERT;
91
92     mqttClient.publish(topic, buffer, true);
93     Serial.println("Publie: " + String(buffer));
94 }
95
96 void setup() {
97     Serial.begin(115200);
98
99     // Configuration GPIO
100     pinMode(PIN_ORANGE, INPUT_PULLUP);
101     pinMode(PIN_ROUGE, INPUT_PULLUP);
102     pinMode(PIN_VERT, INPUT_PULLUP);
103     pinMode(LED_STATUS, OUTPUT);
104
105     // Interruptions
106     attachInterrupt(digitalPinToInterrupt(PIN_ORANGE),
107         handleOrangeButton, FALLING);
108     attachInterrupt(digitalPinToInterrupt(PIN_ROUGE),
109         handleRougeButton, FALLING);
110     attachInterrupt(digitalPinToInterrupt(PIN_VERT),
111         handleVertButton, FALLING);
112
113     // Connexions
114     connectWiFi();
115     mqttClient.setServer(mqtt_server, mqtt_port);
116 }
117
118 void loop() {
119     if (!mqttClient.connected()) {
120         reconnectMQTT();
121     }
122     mqttClient.loop();
123
124     // Traitement evenements avec debounce
125     if (orangePressed) {
126         delay(50);
127         orangePressed = false;
128         publishAlert("DOWNTIME", "Panne");
129     }
130
131     if (rougePressed) {
132         delay(50);
133         rougePressed = false;
134         publishAlert("MAINTENANCE", "Maintenance");
135     }
136 }

```

```

137     if (vertPressed) {
138         delay(50);
139         vertPressed = false;
140         publishAlert("OPERATIONAL", "Resolved");
141     }
142
143     delay(100);
144 }

```

Listing A.1. Firmware ESP32 complet — Configuration et setup

A.2 Annexe B : Schéma Base de Données PostgreSQL

Script complet de création de la base de données avec tables, index et triggers.

```

1  -- Table principale downtime_logs
2  CREATE TABLE downtime_logs (
3      id SERIAL PRIMARY KEY,
4      machine VARCHAR(10) NOT NULL,
5      status VARCHAR(50) NOT NULL,
6      alert_type VARCHAR(100),
7      operator VARCHAR(100),
8      date_panne TIMESTAMP NOT NULL,
9      heure_panne TIME,
10     technician VARCHAR(100),
11     date_arrivee_technicien TIMESTAMP,
12     heure_arrivee TIME,
13     criticite VARCHAR(50),
14     piece_observation TEXT,
15     rfid_uid VARCHAR(50),
16     date_reparation TIMESTAMP,
17     heure_reparation TIME,
18     resolved_by VARCHAR(100),
19     temps_reaction_minutes INTEGER,
20     temps_reparation_minutes INTEGER,
21     temps_total_arret_minutes INTEGER,
22     lifecycle_phase VARCHAR(50) DEFAULT 'detected',
23     atelier VARCHAR(50) DEFAULT 'Test Electrique',
24     zone VARCHAR(10),
25     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
26     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
27 );
28
29 -- Index pour optimisation performances
30 CREATE INDEX idx_downtime_machine ON downtime_logs(machine);
31 CREATE INDEX idx_downtime_date ON downtime_logs(date_panne);
32 CREATE INDEX idx_downtime_status ON downtime_logs(status);
33 CREATE INDEX idx_downtime_phase ON downtime_logs(lifecycle_phase);
34
35 -- Table technicians
36 CREATE TABLE technicians (
37     id SERIAL PRIMARY KEY,
38     name VARCHAR(100) NOT NULL,
39     rfid_uid VARCHAR(50) UNIQUE NOT NULL,
40     specialite VARCHAR(50),
41     email VARCHAR(100),
42     phone VARCHAR(20),
43     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
44 );
45
46 CREATE UNIQUE INDEX idx_technician_rfid ON technicians(rfid_uid);
47
48 -- Table machines
49 CREATE TABLE machines (
50     id SERIAL PRIMARY KEY,
51     code VARCHAR(10) UNIQUE NOT NULL,
52     zone VARCHAR(10) NOT NULL,
53     type VARCHAR(50),
54     description TEXT,
55     status VARCHAR(50) DEFAULT 'operational',
56     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
57 );

```

```

58
59 CREATE UNIQUE INDEX idx_machine_code ON machines(code);
60
61 -- Trigger auto-update updated_at
62 CREATE OR REPLACE FUNCTION update_updated_at_column()
63 RETURNS TRIGGER AS $$
64 BEGIN
65     NEW.updated_at = CURRENT_TIMESTAMP;
66     RETURN NEW;
67 END;
68 $$ language 'plpgsql';
69
70 CREATE TRIGGER update_downtime_updated_at
71 BEFORE UPDATE ON downtime_logs
72 FOR EACH ROW
73 EXECUTE FUNCTION update_updated_at_column();
74
75 -- Insertion donnees exemple technicians
76 INSERT INTO technicians (name, rfid_uid, specialite) VALUES
77 ('Mohamed TAZI', 'A1B2C3D4', 'Electronique'),
78 ('Ali BENNANI', 'E5F6G7H8', 'Mechanique'),
79 ('Fatima ALAMI', 'I9JOK1L2', 'Electricite');
80
81 -- Insertion donnees exemple machines
82 INSERT INTO machines (code, zone, type, description) VALUES
83 ('KA01', 'KA', 'Test continuite', 'Banc test ligne KA'),
84 ('KA02', 'KA', 'Test continuite', 'Banc test ligne KA'),
85 ('KB01', 'KB', 'Test court-circuit', 'Banc test ligne KB'),
86 ('KB02', 'KB', 'Test court-circuit', 'Banc test ligne KB');

```

Listing A.2. Script création base de données PostgreSQL

A.3 Annexe C : Configuration Déploiement

A.3.1 Fichier package.json

```

1 {
2   "name": "andon-system-iiot",
3   "version": "2.1.0",
4   "description": "Systeme Andon Intelligent base sur l'IIoT pour SEWS Maroc",
5   "main": "server.js",
6   "scripts": {
7     "start": "node server.js",
8     "dev": "nodemon server.js"
9   },
10  "dependencies": {
11    "express": "^4.18.2",
12    "socket.io": "^4.6.1",
13    "mqtt": "^5.0.3",
14    "pg": "^8.11.3",
15    "cors": "^2.8.5",
16    "dotenv": "^16.0.3"
17  },
18  "devDependencies": {
19    "nodemon": "^3.0.1"
20  },
21  "engines": {
22    "node": ">=18.0.0",
23    "npm": ">=9.0.0"
24  },
25  "keywords": [
26    "iiot",
27    "iiot",
28    "andon",
29    "industry-4.0",
30    "mqtt",
31    "nodejs",
32    "sews"
33  ],
34  "author": "Aissam MALKOUT",
35  "license": "MIT"
36 }

```

Listing A.3. package.json — Configuration projet Node.js
A.3.2 Variables d'environnement Railway

```

1 # Database
2 DATABASE_URL=postgresql://user:password@host:5432/dbname
3
4 # Server
5 PORT=8080
6 NODE_ENV=production
7
8 # MQTT
9 MQTT_URL=mqtt://broker.hivemq.com:1883
10
11 # Security (optionnel)
12 JWT_SECRET=your_secret_key_here
13 SESSION_SECRET=your_session_secret_here

```

Listing A.4. Variables d'environnement (.env)**A.4 Annexe D : Requêtes SQL Avancées**

Requêtes SQL utilisées pour les analyses et rapports.

```

1 -- KPI par machine
2 SELECT
3     machine,
4     COUNT(*) as total_pannes,
5     AVG(temps_reaction_minutes) as mttm_moyen,
6     AVG(temps_reparation_minutes) as mttr_moyen,
7     AVG(temps_total_arret_minutes) as arret_moyen,
8     MAX(temps_total_arret_minutes) as arret_max
9 FROM downtime_logs
10 WHERE date_panne >= CURRENT_DATE - INTERVAL '30 days'
11 GROUP BY machine
12 ORDER BY total_pannes DESC;
13
14 -- Top 5 pannes les plus frequentes
15 SELECT
16     alert_type,
17     COUNT(*) as occurrences,
18     AVG(temps_total_arret_minutes) as duree_moyenne
19 FROM downtime_logs
20 WHERE date_panne >= CURRENT_DATE - INTERVAL '90 days'
21 GROUP BY alert_type
22 ORDER BY occurrences DESC
23 LIMIT 5;
24
25 -- Performance techniciens
26 SELECT
27     technician,
28     COUNT(*) as interventions,
29     AVG(temps_reparation_minutes) as mttr_moyen,
30     MIN(temps_reparation_minutes) as mttr_min,
31     MAX(temps_reparation_minutes) as mttr_max
32 FROM downtime_logs
33 WHERE technician IS NOT NULL
34     AND date_panne >= CURRENT_DATE - INTERVAL '30 days'
35 GROUP BY technician
36 ORDER BY interventions DESC;
37
38 -- Disponibilite par zone
39 SELECT
40     zone,
41     COUNT(*) as total_pannes,
42     SUM(temps_total_arret_minutes) as temps_arret_total,
43     ROUND(100.0 * (1.0 - (SUM(temps_total_arret_minutes) /
44         (30.0 * 24.0 * 60.0))), 2) as disponibilite_pct
45 FROM downtime_logs
46 WHERE date_panne >= CURRENT_DATE - INTERVAL '30 days'

```

```

47 GROUP BY zone
48 ORDER BY disponibilite_pct ASC;

```

Listing A.5. Requêtes SQL analytiques

A.5 Annexe E : Architecture Réseau



Figure A.1. Architecture réseau et infrastructure de déploiement

Description de l'architecture réseau :

Zone Edge (Atelier) :

- 24 modules ESP32 (un par machine)
- Réseau WiFi industriel 802.11n (SSID : SEWS_WIFI_ATELIER)
- Routeur industriel avec firewall
- Connexion Internet via fibre optique 100 Mbps

Zone Internet :

- Broker MQTT HiveMQ Cloud (broker.hivemq.com :1883)
- Latence moyenne : 30–50 ms
- Disponibilité : 99,9 %

Zone Cloud (Railway) :

- Application Node.js (conteneur Docker)
- PostgreSQL 14 managé
- Load balancer + HTTPS automatique
- Région : Europe (Frankfurt)

A.6 Annexe F : Glossaire Technique Détaillé

ACID Atomicity, Consistency, Isolation, Durability — Propriétés garantissant l'intégrité des transactions en base de données.

Andon Système de management visuel d'origine japonaise permettant aux opérateurs de signaler rapidement les anomalies de production.

API REST Application Programming Interface utilisant le protocole HTTP et respectant les principes REST pour l'exposition de services web.

Broker MQTT Serveur central gérant la distribution des messages MQTT entre publishers et subscribers.

- ESP32** Microcontrôleur 32 bits développé par Espressif Systems, intégrant WiFi, Bluetooth et processeur dual-core 240 MHz.
- GPIO** General Purpose Input/Output — Broches programmables d'un microcontrôleur pouvant servir d'entrées ou sorties numériques.
- IIoT** Industrial Internet of Things — Extension de l'IoT appliquée spécifiquement aux environnements industriels et manufacturiers.
- Jidoka** Principe du Toyota Production System donnant aux opérateurs l'autonomie d'arrêter la production en cas d'anomalie.
- KPI** Key Performance Indicator — Indicateur mesurable permettant d'évaluer la performance d'un processus.
- Latence** Délai entre l'envoi d'un message et sa réception par le destinataire.
- Middleware** Logiciel intermédiaire facilitant la communication entre différents composants d'une application.
- MTTA** Mean Time to Acknowledge — Temps moyen entre la détection d'une panne et l'arrivée du technicien.
- MTTR** Mean Time to Repair — Temps moyen nécessaire pour réparer une panne une fois l'intervention démarrée.
- Node.js** Environnement d'exécution JavaScript côté serveur basé sur le moteur V8 de Chrome.
- Publish/Subscribe** Modèle de communication asynchrone où émetteurs et récepteurs sont découplés via topics.
- PWA** Progressive Web App — Application web utilisant les standards modernes pour offrir une expérience similaire aux applications natives.
- QoS** Quality of Service — Niveau de garantie de livraison des messages dans le protocole MQTT (0, 1 ou 2).
- RFID** Radio Frequency Identification — Technologie d'identification automatique par radiofréquences.
- Socket.IO** Bibliothèque JavaScript facilitant la communication bidirectionnelle temps réel entre clients et serveur.
- TPS** Toyota Production System — Système de production développé par Toyota, fondement du Lean Manufacturing.
- WebSocket** Protocole de communication full-duplex sur une connexion TCP unique, permettant échanges bidirectionnels temps réel.

A.7 Annexe G : Références et Ressources

Documentation technique :

- ESP32 Datasheet : https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- MQTT Specification v5.0 : <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- Node.js Documentation : <https://nodejs.org/en/docs/>
- PostgreSQL Manual : <https://www.postgresql.org/docs/14/>
- Socket.IO Documentation : <https://socket.io/docs/v4/>
- Railway Documentation : <https://docs.railway.app/>

Standards et normes :

- ISO 9001 :2015 — Systèmes de management de la qualité
- IATF 16949 :2016 — Qualité automobile
- ISO 14001 :2015 — Management environnemental
- ISO/IEC 14443 — Cartes RFID sans contact

— IEEE 802.11 — Standards WiFi